Lost & Found

Second Milestone - Software Requirements - INSO 4115

Prof. Marko Schütz-Schmuck

Table of Contents

Notation for Changes

Notation	Purpose	Color	Example
`[.added]# <text> #`	Added	Green	`This text is new.`
`[.changed]# <text> #`	Paraphrased	Orange	`This text has been modified.`
`[.removed]# <text> #`	Removed	Red	`This text is no longer relevant.`

Chapter 1. Informative Part

1.1. Team

The Official Lost and Found (LnF) project is organized into four functional teams and two project managers. All team members listed below are active contributors to the project and are considered confirmed partners because they are directly involved in planning, implementation, documentation, and delivery.

Team members and partners are not always exactly the same. In this project, the confirmed partners currently match the project team members. However, additional external partners may be identified later (for example, course staff or institutional/domain advisors) if requirements clarification or validation is needed.

1.1.1. Confirmed Partners (Current Project Team)

Project Managers

- Iralys Sanchez
- Tomas Gomez

Team 1 - Core System, Base UI, Search and Administration

- Fabian A Acevedo-Martinez (Team Leader)
- Abdiel Cotto-Claudio
- Alberto J Rodriguez-Nogueras
- Diego A Rios-Morales
- Ivaner Bellido-Traverzo
- Joel R Torres-Cruz
- Natanael Batista-Alvarez
- Noel A Colon-Mena

Team 2 - Authentication, Communication and Notifications

- Sebastian G Suarez-Robles (Team Leader)
- Abby Gotay-Almonte
- Angel A Villegas-Colon
- Dylan M Oliver-Martinez
- Fernando A Torres-Delgado
- Javier A Ventosa-Maldonado
- Lianette Alberto-Munoz

Team 3 - Reports and Object Management

- Andres A Cruz-Zapata (Team Leader)
- Adiel Soto-Rodriguez
- Adriel Bracero-Gonzalez
- Jose E Valentin-Cortes
- Naedra J Feliciano-Agostini
- Nelson J Caban-Rodriguez
- Omar A Cordero-Maldonado
- Victoria M Miranda-Osuna

Team 4 - Documentation and Standardization

- Jean P. I Sanchez-Felix (Team Leader)
- Anthony H Martinez-Gomez
- Keishlyany M Sanabria-Santana
- Leanelys A Gonzalez-Orama
- Tasnim Said-Khader

1.1.2. Partner Roles and Responsibilities (Current and Possible)

Current Roles

Role	**Responsibilities**
Project Managers:	Provide overall coordination across teams, monitor progress, ensure consistency between deliverables, and maintain accountability for milestone completion. They also help resolve cross-teams issues and promote quality in both technical and documentation outputs.
Team Leaders:	Organize team-level activities, distribute tasks within team, track progress, and ensure that contributions align with project goals.They are responsible for maintaining consistency within their team's deliverables and communicating dependencies or risks to Project Managers.
Developers:	Carry out assigned tasks, collaborate with teammates, and contribute to implementation, testing, and documentation. They are responsible for producing work that meets agreed quality expectations and for communicating uncertainties or domain-related questions.
Documentation and Standardization Team Members:	Ensure that documentation remains consistent, structured, and aligned with the domain narrative. They support clarity of terminology, maintain formatting, and help ensure that requirements and descriptions remain solution-neutral and traceable.

Possible external partners:

May include course staff, domain advisors, or institutional representatives. Their responsibilities would include the following:

- Provide insight into how lost and found processes operate in real-world contexts.
- Help clarify requirements, validate assumptions, and ensure that domain descpitions accurately reflect practical workflows.
- Offer perspecive on organizational policies or operational constraints, ensuring that responsibilities such as custody, verification and item handling are realistically described.
- Review domain descriptions and requirements for clairty, completeness, and consistency.
- Verify that stakeholder responsibilities and interactions are logically defined and aligned with the core objective of item recovery.

NOTE	Customers or end users of a Lost and Found system are not automatically considered partners. A partner has a stronger and more direct involvement in the project (e.g., coordination, validation, or decision-making). At this stage, no external customer-side partner has been formally identified.

1.2. Current Situation, Needs, Ideas

1.2.1. Current Situation

Informal and Fragmented Lost & Found Practices on Campus

Within the university environment, the handling of lost and found items is largely informal and decentralized. When students lose personal belongings such as laptops, ID cards, wallets, or keys, they typically rely on word of mouth, group chats, social media posts or department offices to attempt recovery. As a result, information about lost or found items is scattered across different channels, making it difficult for affected individuals to know where to look and/or who to contact.

Manual Handling and Limited Traceability

In many cases, found items are turned in to administrative offices. These offices may log items manually or keep handwritten records; however, these records are not publicly searchable and access typically requires visiting the office in person during business hours. Because processes are manual:

- There is limited visibility into which items have already been reported.
- There is no standardized format for describing items.
- There is no automated way to track the lifecycle of an item (e.g., reported, claimed, archived).

This lack of traceability can lead to duplicated efforts, unclaimed items, and unnecessary frustration. [.added]#For example, a student who loses an earbud may recall last using it in the cafeteria, then attending a lecture in a specific classroom, and later studying in the library. To attempt recovery, the student may:#

- [.added]#Return to the cafeteria and ask staff and other students if anything was found#
- [.added]#Visit the classroom after the next lecture to check the area or ask the professor#
- [.added]#Go to nearby administrative offices#
- [.added]#Check with the library front desk or study room office#

Each of these locations operates independently and, as a result, recovering a lost item often requires physically visiting multiple locations with no clear indication of where the item might have been turned in.

Lack of Ownership Enforcement and Accountability

When lost-and-found interactions occur through social media or messaging platforms, there is often no structured mechanism to verify ownership or authenticity. Posts may be edited, deleted, or buried in feeds. There is no reliable way to ensure that only the original poster can modify a report, nor is there a formal moderation process to remove inappropriate or fraudulent listings.

This creates potential risks:

- False claims of ownership.
- Malicious or spam posts.
- Exposure of sensitive personal information.

Inconsistent Identification and Ownership Verification

When items are found and reported informally, there is no consistent method for describing or identifying them. A found item might be described vaguely (e.g., “found keys” or “black wallet”), making it difficult for the rightful owner to confidently identify it. Additionally, when someone claims an item, authentication depends entirely on personal judgment. For example:

- A person claiming a lost phone may only provide a general description
- A finder may not know what details to ask to confirm ownership

There is no shared guideline for verifying claims. This can lead to hesitation in returning items as well as items being claimed by someone other than its rightful owner.

Limited Search and Discovery Capabilities

Students searching for a lost item must manually scroll through posts, check multiple group chats, or visit different offices. There is typically no filtering by category, date, or location, and no consistent use of tags or metadata.

Without structured search capabilities:

- Matching lost and found reports is inefficient.
- Recently reported items may go unnoticed.
- Users cannot sort or filter based on relevant criteria.

Existing Digital Attempts and Their Limitations

Some informal digital attempts exist, such as shared spreadsheets or temporary messaging groups for specific events. While these efforts demonstrate a need for coordination, they lack:

- [.removed]#Authentication and access control.#
- [.removed]#Structured data validation.#
- [.removed]#Persistent lifecycle management.#
- [.removed]#Integrated image handling and secure storage.#

Because these solutions are ad hoc and isolated, they do not provide a sustainable or secure campus-wide solution.

Some informal digital attempts exist, such as messaging groups or shared spreadsheets. These efforts typically arise in specific contexts, such as a class group chat or a temporary group created after an event. While these approaches provide some level of coordination, they tend to be disorganized and inconsistent. For example:

- [.added]#Messages in group chats may contain incomplete or vague descriptions#
- [.added]#Information is quickly buried as new messages are sent#
- [.added]#Posts are not standardized, making it difficult to compare or match items#
- [.added]#Shared documents may lack consistent formatting or may not be regularly updated#
- [.added]#These efforts are often short-lived; once the immediate need or event passes, participation decreases and the information becomes outdated and/or inaccessible#

As a result, although these informal digital methods provide temporary visibility, they do not offer a reliable or sustained way for users to report, track, and recover lost items across the broader campus environment.

1.2.2. Needs

Lost personal belongings such as keys, identification cards, wallets, laptops, and academic materials are a frequent occurrence within the university environment. Despite how common these situations are, there is no clear, consistent, or trusted process that allows individuals to report lost items, discover found belongings, or follow the progress of recovery efforts. As a result, many lost items are never returned to their owners.

Students and staff experience unnecessary stress, frustration, and loss of time when attempting to recover missing belongings. Current informal approaches—such as word of mouth, scattered social media posts, or visiting multiple campus offices—lack visibility, reliability, and coordination. This fragmentation creates uncertainty about whether an item has been found and discourages both reporting and follow-through.

From an organizational perspective, campus entities responsible for handling found items lack a standardized method for recording reports, managing information, and ensuring accountability. The absence of a structured process increases manual effort, introduces communication gaps, and reduces trust in the overall lost-and-found experience.

There is therefore a clear need to support a more transparent, coordinated, and dependable way for the campus community to report, track, and recover lost belongings. Addressing this need would reduce unresolved losses, lower frustration for students and staff, and improve confidence in how lost items are managed within the university environment.

Additionally, stakeholders involved in developing and maintaining such support require a clear understanding of the domain, actors, and operational constraints surrounding lost-and-found activities. Establishing this shared understanding is necessary to define appropriate requirements, ensure alignment with real user needs, and guide the project toward meaningful and sustainable impact.

1.2.3. Ideas

Initial brainstorming focused on identifying practical ways to reduce unresolved lost-item cases and improve visibility across the campus community. The proposed ideas aim to transition from the recognized need for coordination and transparency toward potential forms of support.

Technical Support Concepts

Provide a shared digital space where members of the campus community can report lost and found items. Enable searching or browsing of reported items to help individuals determine whether their belongings have been located. Allow secure communication between individuals involved in a lost-and-found case to coordinate recovery. Use categorization or tagging to improve organization and discovery of reported items. Offer automated alerts when newly reported items resemble previously reported losses. Include administrative oversight to review sensitive or inappropriate reports and maintain trust.

Process and Community Improvements

Encourage consistent reporting behavior across students, staff, and campus offices. Reduce reliance on fragmented communication channels such as informal messages or social media. Improve transparency in how lost items are handled and tracked within the university environment. Promote a culture of responsibility and cooperation around returning found belongings.

Additional or Alternative Considerations

Pilot the approach within a limited campus group before broader adoption. Provide guidance or training for the entity responsible for maintaining the process. Explore integration with existing campus communication or information channels. Consider future expansion to support broader campus safety or notification needs.

These ideas represent early directions rather than finalized requirements. They serve to guide further domain understanding, requirement definition, and evaluation of feasible approaches that meaningfully address the identified needs.

1.3. Scope, Span, and Synopsis

1.3.1. Scope & Span

The Lost and Found project falls under the broad domain of university community support systems and digital resource coordination platforms. Its primary concern is supporting the report and recovery of misplaced or "lost" personal belongings within the University of Puerto Rico Mayagüez (UPRM) campus through a centralized online platform. The scope of this project encompasses:

- Domain analysis, including identification of stakeholders such as students, faculty, and staff at UPRM.
- Requirements engineering, focusing on usability, accessibility, privacy protection, authentication constraints, and misuse prevention.
- System modeling and architectural planning, ensuring that the platform supports secure and structured interaction without entirely replacing existing physical recovery procedures.
- Policy alignment, particularly regarding institutional email restrictions, minimal data exposure, and responsible handling of user-submitted information.

The project is concerned with designing and documenting a platform that facilitates reporting and discovery. It does not aim to manage physical storage of items, mediate disputes directly, or extend beyond the institutional community. Communication between parties occurs outside the system via official institutional email accounts.

While the scope describes the general domain, the span defines the concrete boundaries and operational characteristics of this particular system. The platform is restricted to both students and faculty of the University of Puerto Rico Mayagüez who possess valid institutional email credentials. Its context is limited to campus-related environments such as classrooms, study areas, student centers, and other spaces within the physical boundaries of the UPRM campus. Within this setting, the system specifically supports:

- Submission of lost and found item reports within the UPRM campus.
- Browsing and searching mechanisms that allow users to filter items by category, location, date, etc.
- Controlled visibility of information to prevent unnecessary disclosure of personal details.

The span deliberately excludes broader ambitions such as public access, collaboration with other university campuses within Puerto Rico, integrated messaging systems, financial exchanges, shipment coordination, or automated identity verification beyond institutional email validation.

1.3.2. Synopsis

The Lost and Found project aims to support students and faculty at the University of Puerto Rico Mayagüez by providing a centralized online platform for reporting and discovering misplaced personal belongings within campus. By improving visibility and coordination around lost and found items, the initiative seeks to increase successful recoveries while protecting user privacy and limiting access to the institutional community. The platform focuses solely on facilitating reporting and discovery, without managing physical storage or extending beyond the UPRM campus.

1.4. Other Activities than Just Developing Source Code

Here are several complementary activities that are required to ensure that the system addresses the identified needs and operates effectively within the Lost and Found domain.

1.4.1. Documentation and Knowledge Management

- Maintain comprehensive project documentation aligned with the project outline and evolving design decisions. *Standardize terminology, data fields, and structural conventions across all components.
- Record architectural decisions, workflows, and integration points to support maintainability.
- Keep documentation continuously updated in parallel with development progress.

1.4.2. Domain Engineering

In the current situation, lost and found practices vary across locations and there is no shared or unified understanding of how items are discovered, stored or returned. This inconsistency makes it difficult to describe responsibilities and workflows clearly.

This creates the need for a structured and unified understanding of domain concepts so that all teams base their work on the same assumptions.

DE addresses this need by organizing domain concepts, defining terminology, and establishing a shared narrative. This allows for there to be clarity across teams and ensures that later requirements reflect real-world practices.

1.4.3. Requirements Engineering

The current situation only offers general objectives, such as improving the recovery of lost items, but does not specify what functionalities are required to achieve them, which creates ambiguity for developers and risks inconsistent features.

RE addresses this need by documenting functional expectations, stakeholder goals, and system behaviors. This helps ensure that development efforts align with the objecive of improving the efficiency of lost item recovery.

1.4.4. Software Architecture and Component Design

Several teams work in parallel, and the current situation lacks clearly defined boundaries between responsibilities. Without defined boundaries, parallel development may lead to integration conflicts or duplicated functionality.

Software Architecture and component design addresses this by defining responsibilities for different components, clarifying interfaces, and supporting maintainability and scalability.

1.4.5. Implementation

Currently, lost and found processes are based on informal or disconnected practices, which makes consistent recovery of objects difficult. There is no unified approach for supporting lost item reporting, tracking, and recovery.

This creates the need for concrete functionality that supports stakeholders in managing lost and found items more effectively.

This activity addresses this by trasnforming requirements and designs into working features that support stakeholders in locating and reclaiming lost items efficiently.

1.4.6. Testing and Quality Assurance

The current situation lacks consistent validation of whether proposed workflows actually improve the recovery process. Without proper validation, this could lead to unreliable or confusing interactions.

This creates the need to verify that implemented features support the intended domain processes and meet expected performance indicators.

Testing and quality assurance address this need by verifying functionality, identifying defects, and confirming that performance indicators, like recovery time and matching accuracy, are met.

1.4.7. Deployment and Operational Readiness

Even if functionality is implemented, the current situation may still prevent stakeholders from effectively using the solution due to lack of preparation or guidance. This creates the need to ensure that the solution can be introduced in a way that the stakeholders can realistically adopt.

Deployment planning addresses this by preparing the environment and ensuring that documentation and usage guidance are available.

1.4.8. Version Control and Review Processes

- Use structured version control workflows to manage collaborative development.
- Require peer review before merging major changes.
- Maintain traceability between requirements, implementation changes, and resolved issues.
- Protect stability of the main project branch through controlled integration.

1.4.9. Project Coordination and Communication

- Organize regular team check-ins to monitor progress, blockers, and dependencies.
- Coordinate across functional teams to prevent duplicated work or integration conflicts.
- Ensure alignment between documentation, development, and testing activities.
- Track milestones and deliverables according to the project timeline.

1.4.10. Parallel Execution with Development

Many supporting activities occur simultaneously with coding rather than afterward.

Documentation updates, testing preparation, peer review, and coordination meetings will proceed in parallel with feature implementation to maintain project momentum and reduce late-stage risk. Together, these supporting efforts provide the structure necessary to deliver a reliable, maintainable, and well-coordinated lost-and-found support system for the university community.

1.5. Derived Goals

Reduce Workload for Physical Offices

Due to the platform's primary purpose of logging lost items, physical offices on campus that store lost items should see a decrease in workload regarding lost item management.

Improve UPRM Campus Reputation

The platform, as it currently stands, is limited to the UPRM campus and its members, those being students and faculty. The system being unique to the UPRM campus could increase the campus's reputation for outside members such as highschool students interested in enrolling.

Increase Community Trust within UPRM

As UPRM members see others using the platform to return lost items, these actions become normalized and could foster a culture within campus that makes members feel responsible for helping others recover lost items, increasing the overall trust within the community.

1.5.1. Improve Recovery Outcomes

The primary operational goal is to increase the likelihood that **Lost Items** are successfully returned to their **Owners**. This includes reducing unresolved cases and shortening the average time between **Reporting** and **Recovery** within the UPRM campus context.

1.5.2. Promote Transparency and Trust

The system must foster confidence among students and faculty by ensuring that **Reporting**, **Matching**, and **Closure** processes are clear and accountable. Controlled visibility of information and responsible handling of data are essential to maintaining institutional trust.

1.5.3. Ensure Privacy and Responsible Data Handling

Given the institutional environment and policy alignment requirements, the project must minimize unnecessary data exposure and restrict access to verified UPRM community members. Privacy protection is not only a technical consideration but a guiding project objective.

1.5.4. Maintain Clear Scope Boundaries

The project must remain strictly within its defined span: supporting digital reporting and discovery within UPRM. It deliberately avoids physical item management, dispute mediation, financial transactions, or inter-campus expansion. Maintaining these boundaries prevents scope creep and ensures feasibility.

1.5.5. Prioritize Usability and Accessibility

Because the system serves a diverse academic community, it must be intuitive and accessible. **Reporting** and **Searching** should require minimal effort, encouraging consistent participation and reducing abandonment of recovery attempts.

1.5.6. Support Sustainable Development and Maintainability

The project should be structured to allow future teams to understand, maintain, and extend the platform if needed. Clear documentation, architectural planning, version control practices, and coordinated teamwork are essential supporting goals.

1.5.7. Encourage Collaborative Team Execution

Given the division of responsibilities across multiple teams (authentication, search, reporting, moderation, documentation), coordination and integration are critical goals. Parallel documentation, testing, and review processes must support development rather than follow it.

Chapter 2. Descriptive Part

2.1. Domain Description

2.1.1. Domain Rough Sketches (Raw Observations and Traceable Concept Derivation)

Raw observations (unprocessed)

The table below captures statements and field notes as they were recorded, plus minimal context. If a detail was not captured (for example, what kind of keys), we mark it as unknown rather than guessing.

#	Raw statement / field note	Context (where/when)	Item details (specific)
1	"I left my charger in a classroom. I realized at night. I checked a couple group chats, but nobody replied."	Noticed after hours; checked informal channels	Charger; brand/type unknown; classroom/room not recorded
2	"I lost my keys somewhere between the parking lot and the business building. I wasn't sure if I should check security, the library, or just retrace my steps."	Outdoor route; several plausible places to check	Keys; type not recorded (car, apartment, lab, keycard unknown)
3	"My ID card fell out. I canceled it immediately because I was scared someone would misuse it. Later I found out someone had turned it in to the library."	Defensive action taken before confirmation	Student ID card; details intentionally omitted
4	"I posted on Instagram that I lost my AirPods. Someone replied saying they found some, but it wasn't mine."	Social media used for recovery attempt	Earbuds (AirPods); model/color not recorded
5	"I don't know where the lost and found office is. Back home there's always one central place."	Person unfamiliar with campus expectations	No item specified
6	"I found a phone outside the physics building. I didn't want to leave it there. But I also didn't want to give it to the wrong person."	Item found outdoors; finder hesitant	Phone; brand/model unknown

#	Raw statement / field note	Context (where/when)	Item details (specific)
7	"Students bring water bottles, chargers, USB drives, everything. We put them in a drawer with sticky notes. Sometimes people argue over who it belongs to."	Library handling and storage	Mixed items; sticky note labeling varies; which fields are written is inconsistent
8	"We keep items in the office for a few weeks. If nobody claims them, they get discarded or stored somewhere else. There's no consistent tracking system."	Security office handling	Retention exists ("a few weeks"); exact duration varies or is unknown
9	"I didn't report the calculator I found. I just left it on the desk where I found it."	Passive return attempt	Calculator; model unknown
10	Observation: Unattended items in empty classrooms (notebooks, bottles, jackets). No signage indicating what to do if an item is found.	Classroom building, evening	Items vary; signage absent
11	Observation: Drawer labeled "Lost & Found" contains mixed items with handwritten notes. Notes inconsistent (some dated, some not).	Library third floor office	Note format inconsistent; dates sometimes missing
12	Observation: Keys found near vehicles in a high traffic area; hard to identify ownership.	Parking lot	Keys; type not recorded (metal keys vs keycard unknown)
13	"Anyone lose a blue Hydro Flask?" "Where?" "Near Chem building." "No idea whose it is."	Group chat excerpt	Water bottle (blue Hydro Flask); location vague ("near Chem building")
14	"People come asking 'Has anyone reported a MacBook?' I can't check anything except the drawer."	Library worker perspective	MacBook; no searchable record (only physical drawer)
15	"I wouldn't put my phone number publicly. I don't want random people contacting me."	Privacy boundary in open reporting	No item specified
16	Observation: People retrace steps first; only later ask peers/offices.	Multiple informal discussions	No item specified
17	"I assumed someone would steal it if I didn't find it quickly."	Perceived risk affecting urgency	No item specified

Evidence add on (questionnaire summary)

- Respondents: N = 10 (8 students, 2 security staff)
- Collection method: Google Form
- Dates: March 16, 2026 to April 3, 2026
- Limitations:
    - Small sample size
    - Convenience sample (mostly within our network)
    - Limited representation of staff perspectives

Key results:

- 8/10 reported losing at least one item in the last 6 months.
- Common items mentioned: water bottles, chargers, earbuds, keys.
- Median number of places checked before finding or giving up: 2.
- Places and channels commonly checked: library, department offices, group chats.
- Privacy boundaries were common (contact info, ID details, exact locations).
- 3/10 mentioned concerns about suspicious or false claiming behavior.
- Most respondents reported noticing the loss later the same day (often within a few hours).

Derived concepts (traceable back to raw observations)

The concepts below are derived from the raw observation table above. Each concept includes the observation numbers it is based on.

Derived concept	**What it means**	**Supported by obs #**
Delayed realization	People often notice loss later (hours later or after hours), not immediately.	1, 16
No clear starting point	After noticing a loss, people are unsure which office or place is the right first stop.	2, 5, 14
Place to place effort	Recovery attempts require checking multiple places and channels.	2, 14, 16
Fragmented channels	People use informal channels (social media and chats) that do not preserve context well.	4, 13
Weak matching / false positives	Informal matching creates false positives and uncertainty.	4
Finder hesitation	Finders want to help but fear giving the item to the wrong person.	6
Manual custody	Items end up stored in offices or drawers with inconsistent notes and metadata.	7, 11, 14

Derived concept	What it means	Supported by obs #
Disputes at pickup	Weak verification leads to arguments over ownership.	7
Retention window	Items are held for some time, then discarded or moved.	8
Non reporting / passive return	Some finders do not report; they leave items where found.	9
Privacy boundary	People avoid posting personal contact details publicly.	15
Theft anxiety / urgency escalation	Fear of theft increases urgency and changes behavior.	17

Missing specifics to collect next

We intentionally did not invent details that were not captured in the raw observations. Next time we collect observations, we will explicitly capture:

- Keys: car keys vs apartment keys vs lab keys vs keycard (obs #2 and #12).
- Earbuds: model, color, case markings (obs #4).
- Chargers: USB C vs Lightning vs laptop charger block and cable (obs #1).
- Retention: approximate duration range and whether it differs by office (obs #8).

2.1.2. Terminology

This section defines key concepts within the lost-and-found domain. Each term is annotated with its **type** (entity, event, state, behavior, function) and the **phase** in which it is introduced.

Type: Entity & Phase: Domain

Term	Definition
Lost Item	Personal belonging whose owner no longer knows its location.
Found Item	Personal belonging discovered by someone other than its owner.
Owner	Individual who has legitimate claim over a lost item.
Finder	Individual who discovers a lost item.
Holder	A person or office temporarily responsible for keeping a found item.
Loss Location	Area where an item is believed to have been lost.

Term	Definition
Discovery Location	Area where an item is found, which may differ from the loss location.
Custody Location	The place where a found item is kept while awaiting return.
Report	A recorded description of a lost or found item used to support identification and recovery.

Type: State & Phase: Domain

Term	Definition
Holding Period	The duration during which a found item remains under custody before alternative handling is considered.
Unclaimed	State of a found item after the holding period expires without successful return.
Active Report	State in which a report is open and recovery is still expected.
Closed Report	State in which no further recovery actions are expected.

Type: Event & Phase: Domain

Term	Definition
Loss Realization	The moment an individual becomes aware that an item is missing.
Item Found	The moment an item is discovered by someone other than its owner.
Item Handed Over	The transfer of a found item to a holder or custody location.
Item Reported	An item is formally or informally documented as lost or found.
Claim Attempt	An individual asserts ownership of a found item.
Item Returned	The event in which a found item is successfully transferred back to its owner.
Holding Period Expired	The moment when a defined holding period ends without successful return.
Report Closed	The event marking the conclusion of a report's lifecycle.

Type: Behavior & Phase: Domain

<table>
<tr><th>Term</th><th>Definition</th></tr>
<tr><td>Item Handling</td><td>Behavior composed of:
Event: Item Found
Event: Item Handed Over
State: Item stored at Custody Location
Event: Claim Attempt (optional)
Event: Item Returned or Holding Period Expired
This behavior models how a found item transitions from discovery to resolution.</td></tr>
<tr><td>Claim Verification</td><td>Behavior composed of:
Event: Claim Attempt
Action: Comparison of identifying details
Outcome:
Event: Item Returned (if verified)
No return (claim denied)
This behavior ensures that items are returned only to legitimate owners.</td></tr>
<tr><td>Matching</td><td>Behavior composed of:
Event: Item Reported
Action: Comparison between Lost Item and Found Item descriptions
Event: Claim Attempt (if a potential match is identified)
This behavior represents how potential matches are identified.</td></tr>
<tr><td>Custody Management</td><td>Behavior composed of:
Event: Item Handed Over
State: Item stored at Custody Location
State: Holding Period active
Event:
Item Returned
or Holding Period Expired → State: Unclaimed
This behavior models how items are managed while in custody.</td></tr>
</table>

Type: Function & Phase: Requirements

Term	Definition
Reporting	System-supported action of documenting a lost or found item in a structured and persistent manner.
Notification	Mechanism used to inform users about relevant events such as matches or status changes.
Authentication	Mechanism used to confirm the identity of a user before allowing restricted actions.

2.1.3. Narrative

Within the university environment, personal belongings frequently change hands unintentionally. A student may leave a backpack in a classroom, misplace keys in a laboratory, or forget an ID card in the library. In each of these situations, the items becomes a **Lost Item** from the perspective of its **Owner**. The owner's primary concern is retrieving the belonging safely and efficiently.

At the some point, another individual may encounter the object. This person becomes the **Finder**. After discovering the item, the **Finder** may keep it temporarily, transfer it to another individual, or bring it to a known **Custody Location**. These decisions influence how visible the **Found Item** becomes to the Owner.

Communication between **Owners**, **Finders**, **Holders** occur through existing channels within the university environment. These may include:

- direct interpersonal communication (asking classmates or colleagues),
- physical notices in common areas,
- communication through instructors or staff, and
- inquiries at offices that temporarily hold found items.

These communication channels vary in reach, reliability, and timeliness. For example, direct communication within a small group can be quick but limited in scope, while physical notices can reach more people but go unnoticed by the owner. Consequently, the effectiveness of communication influences the likelihood that a **Lost Item** and a **Found Item** are related.

Several factors affect the probability of an item being returned:

- whether the **Finder** takes it to a recognized safekeeping facility,
- how quickly the owner realizes the loss,
- the clarity of the item's identifying characteristics,
- the overlap between the communication circles of the owner and the finder,
- the length of time the item remains available for retrieval, and
- the accessibility of the storage location.

When information about a **Lost Item** and a **Found Item** is available, the possibility of comparison arises. Comparison involves comparing the descriptions to determine if they refer to the same physical object. If a plausible association is established, a **Claim Attempt** may occur, followed by verification, where ownership is assessed using identifying data. If verification is successful, the item is returned to its **Owner**. In other cases, the item may remain in custody until the retention period expires, after which it is considered unclaimed. Throughout this process, ownership is characterized by events such as loss, discovery, communication, temporary custody, claims attempts, and ultimately, return or expiration.

2.1.4. Events, Actions, and Behaviors

This section integrates domain events, actions, and typical behaviors into the Ubiquitous Language for the campus Lost & Found domain. All entries are system-independent, classified strictly as Event or Action, and linked to domain entities. Scenario analysis is included to surface missing or implied domain concepts.

Behavioral Patterns

While events are instantaneous and actions are discrete, behaviors describe structured sequences unfolding over time within the campus Lost & Found environment. They capture typical user workflows and system interactions, such as reporting, searching, claiming, verifying, and moderating. Each behavior is defined by its constituent events and actions, expected outcomes, and reference to real-world scenarios.

User authenticated (event / operation)

- Type: hybrid — operation (authentication attempt) and event (session established)
- Label: user just authenticated
- Triggered by: University member logging in via institutional SSO.
- Immediate response: Authenticated session created; user identity bound to session context.
- Expected outcome/postcondition: User gains access to protected actions (submit, edit, resolve, flag).
- Reference to sketch: Platform restricted to verified university community members.

Lost item report submitted (event / action)

- Type: hybrid — action (submission) and event (report recorded)
- Label: lost item report just published
- Triggered by: Authenticated user submitting structured lost item form.
- Immediate response: Input validated; report stored with `status = "active"`; ownership bound to user_id.
- Expected outcome/postcondition: Report becomes visible in searchable listings.
- Reference to sketch: Students describe posting lost laptops, IDs, electronics.

Found item report submitted (event / action)

- Type: hybrid — action (submission) and event (report recorded)
- Label: found item report just published
- Triggered by: Authenticated user submitting structured found item form.
- Immediate response: Input validated; report stored with `status = "active"`; ownership assigned.
- Expected outcome/postcondition: Listing becomes publicly searchable for potential claimants.
- Reference to sketch: Community members report found wallets and IDs.

Report updated (event / operation)

- Type: hybrid — operation (edit) and event (modification recorded)
- Label: report just modified
- Triggered by: Owner editing description, location, images, or metadata.
- Immediate response: Updated fields persisted; modification timestamp refreshed.
- Expected outcome/postcondition: Search results and visible details reflect new information.
- Reference to sketch: Users correcting typos or adding clarifying details.

Claim contact initiated (event / action)

- Type: hybrid — action (contact initiation) and event (claim request recorded)
- Label: claim request just initiated
- Triggered by: User contacting reporter regarding a listing.
- Immediate response: Institutional email channel opened or contact metadata recorded.
- Expected outcome/postcondition: External verification conversation begins.
- Reference to sketch: Owners contacting finders after discovering matching listing.

Report marked resolved (event / operation)

- Type: hybrid — operation (status change) and event (resolution recorded)
- Label: report just resolved
- Triggered by: Owner marking listing as completed after recovery.
- Immediate response: Lifecycle state updated from `"active"` to `"resolved"`.
- Expected outcome/postcondition: Listing excluded from active search results.
- Reference to sketch: Students closing listings after item recovery.

Report flagged for review (event / action)

- Type: hybrid — action (flagging) and event (flag recorded)
- Label: report just flagged
- Triggered by: User identifying suspicious or inappropriate listing.

- Immediate response: Flag metadata recorded; administrative review process notified.
- Expected outcome/postcondition: Listing enters moderation workflow.
- Reference to sketch: Concerns about spam or malicious reports.

Report removed by administrator (event / operation)

- Type: hybrid — operation (removal) and event (removal recorded)
- Label: report just removed
- Triggered by: Administrator enforcing policy violation.
- Immediate response: Lifecycle state set to `"removed"`; listing hidden from public feed.
- Expected outcome/postcondition: Platform integrity preserved.
- Reference to sketch: Oversight behavior ensuring trust within system.

Reporting behavior

- Type: composite behavioral sequence
- Label: user reporting workflow

Typical progression:

1. Authenticate → **User authenticated**
2. Submit lost/found form → **Lost item report submitted / Found item report submitted**
3. Optional edit → **Report updated**
4. Listing remains `"active"` until resolved or removed

Expected outcome: Structured, searchable representation of real-world lost/found item.

Reference to sketch: Community members formalize informal notice-board behavior through digital submission.

Search and claim behavior

- Type: composite behavioral sequence
- Label: recovery attempt workflow

Typical progression:

1. Authenticate → **User authenticated**
2. Search/filter listings (query operation; no event)
3. Identify potential match
4. Initiate contact → **Claim contact initiated**

Expected outcome: Off-platform verification conversation begins.

Reference to sketch: Students repeatedly checking listings to locate missing belongings.

Verification and return behavior

- Type: composite behavioral sequence
- Label: recovery completion workflow

Typical progression:

1. Claim contact initiated
2. External identity verification between users
3. Owner marks listing resolved → **Report marked resolved**

Expected outcome: Lifecycle transition from `"active"` to `"resolved"`; search space reduced.

Reference to sketch: Users confirming identity before returning wallet or device.

Oversight and moderation behavior

- Type: composite behavioral sequence
- Label: trust preservation workflow

Typical progression:

1. User flags suspicious listing → **Report flagged for review**
2. Administrative evaluation
3. Admin removes listing if necessary → **Report removed by administrator**

Expected outcome: Platform integrity maintained; inappropriate content removed.

Reference to sketch: Need for moderation to prevent misuse or fraudulent claims.

This revised structure distinguishes clearly between:

- Operations (intentional user/system actions),
- Events (state changes recorded in the domain),
- Composite behaviors (temporal interaction patterns).

It aligns domain concepts such as **Report**, **Ownership**, and **Lifecycle State** with concrete operational flows, ensuring structural consistency across terminology, requirements, and implementation.

Domain Events

Events are observable facts that have occurred in the domain. They are immutable records of domain reality (what happened), not prescriptions for how a system must implement them.

Event	Definition	Trigger	Observable Condition / Resulting State
Lost item reported	A member declares that an identifiable item has been lost.	A member provides a description, approximate location, and time of loss.	A Lost Item report exists with status **active**; report contains description, time, and location.
Found item reported	A member declares that an identifiable item has been found.	A member provides a description of the found item, location found, and time found.	A Found Item report exists with status **active**; report contains description, location, and time.
Claim initiated	A member expresses intent to recover an item associated with a report.	A member contacts the reporter or otherwise signals intent to claim/return the item.	A claim record or contact metadata exists; an off-platform or in-person recovery process begins.
Claim verified	The parties have established sufficient evidence or mutual confirmation that the claimant is the rightful owner.	Parties perform identity/ownership confirmation (e.g., matching unique details, in-person ID check).	A verification note or record exists indicating verification succeeded; parties proceed to transfer.
Item returned	Physical possession of the item transfers from the finder to the owner.	Finder and owner complete a physical handover.	A return record exists; custody of the item is with the owner; associated report may be updated.
Report resolved	The matter associated with a report is considered closed (item returned, owner gives up, or other resolution).	Owner or finder declares closure, or parties agree resolution through other means.	Report status changes to **resolved**; no further recovery actions are expected.
Report challenged	A member raises a domain-level concern about a report (suspicion of fraud, duplicate, or inappropriate content).	A member identifies an issue and communicates the concern to an oversight actor.	A challenge record exists; the report enters a review workflow.
Report removed	A report is permanently withdrawn from public circulation due to policy or community enforcement.	An authorized reviewer determines removal is warranted after review.	Report status changes to **removed**; the report is no longer visible to the community.

Domain Actions

Actions are intentional activities performed by domain actors. Actions are distinct from events: actions describe what actors do; events describe what is observed.

Action	Actor	Description	Primary Related Entities
Submit lost report	Member	Provide a factual description of a lost item (what, where, when) and declare the loss to the community.	Member; LostItemReport
Submit found report	Member	Provide a factual description of a found item (what, where, when) and declare the finding to the community.	Member; FoundItemReport
Update report	Member	Amend or add details to an existing report (e.g., additional identifying marks, corrected location).	Report; Member
Initiate claim	Member	Express intent to recover an item by contacting the reporter or otherwise signaling a claim.	Claim; Member; Report
Verify claim (external identity verification)	Members involved in a claim	Confirm identity or ownership through domain-level means (in-person ID, unique item detail, institutional credential).	Claim; Member
Transfer item custody (handover)	Finder and Owner	Physically hand the item from finder to owner, including any agreed verification steps.	Item; Member
Mark resolved	Owner or Finder	Declare that the report's matter is closed (item returned, owner withdraws claim, or other agreed resolution).	Report; Member
Challenge report	Member	Raise a domain concern about a report's legitimacy or content.	Report; Challenge; Member
Review challenge	Administrator or designated reviewer	Examine a challenged report and decide whether to clear the challenge or remove the report.	Report; Challenge; Administrator
Remove report	Administrator or designated reviewer	Permanently withdraw a report from community circulation following review and policy enforcement.	Report; Administrator

Behaviors (Scenarios)

Behaviors are composite sequences of actions and events that represent typical domain workflows.

Each step maps to a defined domain action or event. These scenarios are system-independent: they describe what happens in the community, not how a system must implement it.

Reporting an Item

1. Action: Submit lost report or Submit found report
2. Event: Lost item reported or Found item reported
3. Action (optional): Update report

Outcome: A public domain report exists and is discoverable by other members.

Notes: Reporting actors are responsible for the accuracy of details. Updates are domain actions that produce observable modifications.

Finding and Claiming an Item

1. Action: Member searches available reports (informational activity; not an event)
2. Action: Initiate claim
3. Event: Claim initiated
4. Action: Verify claim (external identity verification)
5. Event: Claim verified
6. Action: Transfer item custody (handover)
7. Event: Item returned
8. Action: Mark resolved
9. Event: Report resolved

Outcome: Item custody transfers to the rightful owner and the report is closed.

Notes: Verification is a social/domain action that may use institutional credentials or unique item details; it is outside system enforcement unless the community defines a domain-level verification protocol.

Return and Resolution without Handover

1. Action: Owner locates item independently (not via the finder)
2. Action: Owner marks resolved
3. Event: Report resolved

Outcome: Report closes without an Item returned event involving the finder. This scenario distinguishes the resolution from the physical return.

Moderation and Trust Management

1. Action: Challenge report
2. Event: Report challenged

3. Action: Review challenge
4. If valid: Action: Clear challenge → Event: Flag cleared (report remains active)
5. If invalid: Action: Remove report → Event: Report removed

Outcome: Community integrity is preserved through domain review and enforcement.

Conceptual Function Signatures

Purpose: These conceptual functions describe the core domain operations from a system-independent perspective, including their purpose, required inputs, outputs, and dependencies. They map to the domain events and actions defined above.

Authenticate User

```
authenticateUser(email: UPREmail, token: AuthToken): AuthenticatedSession
```

- **Description:** Verifies a user's credentials using the official UPR authentication system and establishes an authenticated session.
- **Conceptual Inputs:** UPR email, UPR authentication token.
- **Conceptual Output:** Authenticated session (represents the user's logged-in state).
- **Dependencies:** Relies on the UPR identity provider for credential validation.
- **Related Domain Events/Actions:** This function enables all authenticated actions; it is a prerequisite for **Submit lost report**, **Submit found report**, **Update report**, **Initiate claim**, **Challenge report**, etc.

Authorize Action

```
authorizeAction(user: Person, action: Action): AuthorizationDecision
```

- **Description:** Determines whether a user is permitted to perform a specific action (e.g., creating a report, moderating content) based on their role.
- **Conceptual Inputs:** User, requested action.
- **Conceptual Output:** Permission decision (allowed/denied).
- **Dependencies:** Depends on the user's assigned role and the system's authorization rules.
- **Related Domain Events/Actions:** Used to enforce permissions for actions such as **Modify Report**, **Mark resolved**, **Review challenge**, **Remove report**.

Register Lost Item

```
registerLostItem(lost: Item, reporter: Person, lostAt: Location, lostOn: DateTime,
category: Category): LostItemReport
```

- **Description:** Creates a report for an item that has been lost. The report is initially marked as active.

- **Conceptual Inputs:** Reporter (user), item description, location where lost, date/time lost, category.
- **Conceptual Output:** Lost item report.
- **Dependencies:** Requires the user to be authenticated and the category to exist.
- **Related Domain Events/Actions:** Corresponds to **Submit lost report** (action) and **Lost item reported** (event).

Register Found Item

```
registerFoundItem(found: Item, finder: Person, foundAt: Location, foundOn: DateTime,
category: Category): FoundItemReport
```

- **Description:** Creates a report for an item that has been found. The report is initially marked as active.
- **Conceptual Inputs:** Finder (user), item description, location found, date/time found, category.
- **Conceptual Output:** Found item report.
- **Dependencies:** Requires the user to be authenticated and the category to exist. The finder may be optional depending on the domain rules.
- **Related Domain Events/Actions:** Corresponds to **Submit found report** (action) and **Found item reported** (event).

Modify Report

```
modifyReport(report: Report, updatedDetails: ReportUpdate): Report
```

- **Description:** Updates the details of an existing report while preserving data integrity.
- **Conceptual Inputs:** Report, updated details (the fields to change).
- **Conceptual Output:** Updated report.
- **Dependencies:** Only the original author or an administrator may modify a report.
- **Related Domain Events/Actions:** Corresponds to **Update report** (action). The resulting change may be observed as an update to the report state.

Change Report Status

```
changeReportStatus(report: Report, newStatus: ReportStatus): Report
```

- **Description:** Transitions a report through its lifecycle states (e.g., active, returned, closed).
- **Conceptual Inputs:** Report, new status.
- **Conceptual Output:** Updated report.

- **Dependencies:** Status changes may be restricted based on the user's role and the current report state.
- **Related Domain Events/Actions:** Corresponds to **Mark resolved** (action) and may also apply to **Report removed** (event) when status changes to removed.

Verify Ownership

```
verifyOwnership(foundReport: FoundItemReport, claimant: Person, verificationDetails:
VerificationData): VerificationResult
```

- **Description:** Confirms that a claimant is the rightful owner of a found item by comparing private verification details (e.g., serial numbers, unique identifiers) with those stored in the report.
- **Conceptual Inputs:** Found report, claimant (user), verification details.
- **Conceptual Output:** Verification result (success/failure).
- **Dependencies:** Requires the verification details to be provided by the claimant; may trigger a status change or reward handling if successful.
- **Related Domain Events/Actions:** Corresponds to **Verify claim (external identity verification)** (action) and **Claim verified** (event).

Search Reports

```
searchReports(keywords: SearchQuery): ReportList
```

- **Description:** Finds reports that match a set of search keywords, covering both lost and found items.
- **Conceptual Inputs:** Search keywords.
- **Conceptual Output:** List of matching reports.
- **Dependencies:** The search may be performed by any user without authentication.
- **Related Domain Events/Actions:** This is an informational activity referenced in the **Finding and Claiming an Item** behavior; it does not directly correspond to a domain event or action in the tables.

Filter Reports

```
filterReports(status: ReportStatus, category: Category, location: Location, dateRange:
DateRange): ReportList
```

- **Description:** Narrows search results by applying criteria such as status, category, location, or date range.
- **Conceptual Inputs:** Status, category, location, date range.
- **Conceptual Output:** List of filtered reports.

- **Dependencies:** Can be combined with a keyword search to refine results further.
- **Related Domain Events/Actions:** Like search, this is an informational activity supporting discovery.

Send Message

```
sendMessage(sender: Person, report: Report, content: MessageContent): Message
```

- **Description:** Allows a user to contact another user regarding a specific report without exposing personal contact details. The system mediates the communication.
- **Conceptual Inputs:** Sender (user), report, message content.
- **Conceptual Output:** Message record (stored for audit and delivery).
- **Dependencies:** Both sender and recipient must be authenticated; the message is tied to a report.
- **Related Domain Events/Actions:** This function supports the **Initiate claim** action by providing a mediated communication channel.

Flag Report

```
flagReport(report: Report, flaggedBy: Person, reason: FlagReason): FlagRecord
```

- **Description:** Enables a user to report potentially inappropriate or suspicious content for administrative review.
- **Conceptual Inputs:** Report, user flagging, reason.
- **Conceptual Output:** Flag record.
- **Dependencies:** Flagging does not automatically remove content; it creates a record for later moderation.
- **Related Domain Events/Actions:** Corresponds to **Challenge report** (action) and **Report challenged** (event).

Moderate Report

```
moderateReport(report: Report, admin: Person, action: ModerationAction): Report
```

- **Description:** Allows an administrator to review a flagged report and take action (e.g., edit, remove, approve).
- **Conceptual Inputs:** Report, administrator, action to perform.
- **Conceptual Output:** Updated report or removal confirmation.
- **Dependencies:** Only users with the administrator role may perform moderation actions.
- **Related Domain Events/Actions:** Corresponds to **Review challenge** (action). If removal is chosen, it leads to **Report removed** (event).

Categorize Report

```
categorizeReport(report: Report, category: Category): Report
```

- **Description:** Assigns a category from the standardized taxonomy to a report, aiding in organization and filtering.
- **Conceptual Inputs:** Report, category.
- **Conceptual Output:** Updated report.
- **Dependencies:** The category must exist in the system's taxonomy.
- **Related Domain Events/Actions:** This is part of **Submit lost report** and **Submit found report** actions, and may be updated via **Update report**.

Notify User of Match

```
notifyUserOfMatch(user: Person, matchedReport: Report): Notification
```

- **Description:** Generates an alert for a user when the system detects a potential match between a lost report and a found report.
- **Conceptual Inputs:** User, matched report.
- **Conceptual Output:** Notification (sent to the user).
- **Dependencies:** The match detection is performed by the system based on configurable criteria.
- **Related Domain Events/Actions:** This function is a system-initiated action; it does not correspond directly to a domain event or action in the tables but supports the discovery workflow.

Compute Return Rate

```
computeReturnRate(dateRange: DateRange, category: Category, location: Location):
ReturnRate
```

- **Description:** Calculates the percentage of items successfully returned relative to the total number of resolved reports.
- **Conceptual Inputs:** None.
- **Conceptual Output:** Return rate percentage.
- **Dependencies:** Operates on aggregate report data and is typically used for administrative dashboards.
- **Related Domain Events/Actions:** This is an analytical function that uses the **Report resolved** events and **Item returned** events to compute the metric.

2.1.5. Function Signatures

User Access & Authentication

1. Authenticate with UPR

Event / Trigger: User attempts to log in using their official `upr.edu` account.

System Behavior: - Validates credentials through UPR authentication integration. - Creates an authenticated session upon success.

Inputs: - `upr_email: str` — User's official UPR email address - `upr_token: str` — Authentication token issued by UPR

Output: - `AuthSession`

Function Signature:

```
def authenticate_with_upr(upr_email: str, upr_token: str) -> AuthSession
```

2. Authorize User Action

Event / Trigger: User attempts to perform a restricted action (e.g., reporting an item).

System Behavior: - Verifies whether the user's role permits the requested action.

Inputs: - `user_id: str` - `action: str`

Output: - `bool`

Function Signature:

```
export async function authorizeUserAction(userId: string, action: string) -> bool
```

Report Creation & Management

3. Create Lost Report

Event / Trigger: User submits the Lost Item form.

System Behavior: - Validates required fields. - Saves report. - Sets status to "Active".

Inputs: - `user_id: str` - `report_data: dict`

Output: - `Report`

Function Signature:

```
export async function createLostReport(userId: string, reportData: dict) -> Report
```

4. Create Found Report

Event / Trigger: User submits the Found Item form.

System Behavior: - Validates required fields. - Saves report. - Sets status to "Active".

Inputs: - `user_id: str` - `report_data: dict`

Output: - `Report`

Function Signature:

```
export async function createFoundReport(userId: string, reportData: dict) -> Report
```

5. Edit Report

Event / Trigger: User edits an existing report.

System Behavior: - Updates report fields. - Preserves data integrity.

Inputs: - `report_id: str` - `updated_data: dict`

Output: - `Report`

Function Signature:

```
export async function editReport(reportId: string, updatedData: dict) -> Report
```

6. Update Report Status

Event / Trigger: User or admin changes report status (Active / Returned / Closed).

System Behavior: - Updates report lifecycle state.

Inputs: - `report_id: str` - `new_status: str`

Output: - `Report`

Function Signature:

```
export async function updateReportStatus(reportId: string, newStatus: string) ->
Report
```

Ownership Verification

7. Validate Item Ownership

Event / Trigger: User claims ownership of a found item.

System Behavior: - Compares private verification fields (e.g., serial number, identifying details). -

Allows status change only if validation succeeds.

Inputs: - `report_id: str` - `claimant_id: str` - `verification_data: dict`

Output: - `bool`

Function Signature:

```
export async function validateItemOwnership(
    reportId: string,
    claimantId: string,
    verificationData: dict
) -> bool
```

Search & Discovery

8. Search Reports by Keyword

Event / Trigger: User enters search keywords.

System Behavior: - Returns matching lost or found reports.

Inputs: - `query: str`

Output: - `list[Report]`

Function Signature:

```
export async function searchReportsByKeyword(query: string) -> list[Report]
```

9. Filter Reports

Event / Trigger: User applies filters (status, category, location, date range).

System Behavior: - Returns filtered report results.

Inputs: - `status: str` - `category: str` - `location: str` - `date_range: tuple`

Output: - `list[Report]`

Function Signature:

```
export async function filterReports(
    status: string,
    category: string,
    location: string,
    date_range: tuple
) -> list[Report]
```

Communication System

10. Send Secure Message

Event / Trigger: User contacts another user regarding a report.

System Behavior: - Mediates communication internally. - Prevents exposure of private email addresses.

Inputs: - `sender_id: str` - `report_id: str` - `message: str`

Output: - `Message`

Function Signature:

```
export async function sendSecureMessage(
    senderId: string,
    reportId: string,
    message: string
) -> Message
```

Moderation & Admin System

11. Flag Report

Event / Trigger: User flags inappropriate or suspicious content.

System Behavior: - Records flag for administrative review.

Inputs: - `report_id: str` - `flagged_by: str` - `reason: str`

Output: - `Flag`

Function Signature:

```
export async function flagReport(reportId: string, flaggedBy: string, reason: string)
-> Flag
```

12. Moderate Report

Event / Trigger: Administrator reviews flagged report.

System Behavior: - Edits, removes, or approves report content.

Inputs: - `report_id: str` - `admin_id: str` - `action: str`

Output: - `Report`

Function Signature:

```
export async function moderateReport(reportId: string, adminId: string, action: string) -> Report
```

Categorization System

13. Assign Category to Report

Event / Trigger: User selects category during report creation.

System Behavior: - Links report to standardized taxonomy category.

Inputs: - `report_id: str` - `category_id: str`

Output: - `Report`

Function Signature:

```
export async function assignCategoryToReport(reportId: string, categoryId: string) -> Report
```

Notification System

14. Notify User of Match

Event / Trigger: System detects a potential match between lost and found reports.

System Behavior: - Generates a notification alert for the user.

Inputs: - `user_id: str` - `matched_report_id: str`

Output: - `None`

Function Signature:

```
export async function notifyUserOfMatch(userId: string, matchedReportId: string) -> None
```

Dashboard & Analytics

15. Calculate Return Success Rate

Event / Trigger: Administrator views dashboard analytics.

System Behavior: - Computes percentage of returned items relative to total reports.

Inputs: - None

Output: - `float`

Function Signature:

```
export async function calculateReturnSuccessRate() -> float
```

2.2. Requirements

2.2.1. User Stories, Epics, Features

Abbreviations and ID Conventions:

Abbreviation	Meaning
US	User Story: a user-centered need framed as intent and value.
E	Epic: a high-level goal that groups related features and stories.
F	Feature: a concrete capability that realizes part of an epic.
ReqRef	Requirement Reference: the requirement ID(s) a story or feature maps to.
REQ	Requirement: a functional or non-functional specification with a stable ID.

Identifier Formats:

Type	Format	Components	Example
User Story ID:	`US.AREA.NN`	`AREA` = functional area (`AUTH`, `REPORT`, `SEARCH`, `CLAIM`, `ADMIN`, `UX`)	`US.REPORT.01`
Epic ID:	`E#` or "Epic E#: Title"	`#` = epic number	`E1`
Feature ID:	`F#.N`	`#` = epic number; `N` = feature sequence	`F1.1`
Requirement ID:	`REQ-AREA-TYPE-NN`	`TYPE` = `F` (Functional) or `NF` (Non-functional)	`REQ-REPORT-F-01`

Epic E1: Reporting Lost and Found Items

Goal: Provide a structured and reliable way for users to report lost or found items within the campus environment.

Problem/value: Without a structured reporting mechanism, lost and found items are handled informally, increasing recovery time and confusion.

Features (F1):

- F1.1 Lost item submission
- F1.2 Found item submission

- F1.3 Image upload support
- F1.4 Edit and manage own reports
- F1.5 Automatic ownership binding (user-session association)

US.REPORT.01: Submit a lost item report | ReqRef: [REQ-REPORT-F-01]

"As a user who lost an item, I want to report it lost so that others can help locate it."

Acceptance criteria:

Given	When	Then
Valid report data	Form is submitted	Report is created with status "active".
Missing required fields	Submission attempted	Validation errors are displayed.

US.REPORT.02: Submit a found item report | ReqRef: [REQ-REPORT-F-02]

"As a user who found an item, I want to upload photos of it so that the owner can identify it."

Acceptance criteria:

Given	When	Then
Valid data and optional image	Form submitted	Report and image references are stored.

E1 Traceability (Stories → Features → Requirements):

Story ID	Feature	ReqRef	Notes
US-REPORT-01	F1.1, F1.5	[REQ-REPORT-F-01]	Ownership bound to session.
US-REPORT-02	F1.2, F1.3	[REQ-REPORT-F-02]	Image storage separation.

Epic E2: Search and Discovery of Reports

Goal: Allow users to efficiently locate relevant listings.

Problem/value: Users need quick filtering and search capabilities to reduce stress and time spent locating items.

Features (F2):

- F2.1 Category filtering
- F2.2 Keyword search
- F2.3 Sorting by date
- F2.4 Clear Lost/Found labeling
- F2.5 Status-based filtering (active/resolved)

US.SEARCH.01: Filter reports by category | ReqRef: [REQ-SEARCH-F-01]

"As a user who lost an item, I want to filter listings by category so that I can narrow down results and find my missing item."

Acceptance criteria:

Given	When	Then
Reports exist	Category selected	Only matching reports are displayed.

US.SEARCH.02: Search by keyword | ReqRef: [REQ-SEARCH-F-02]

"As a user who lost an item, I want to filter listings by category so that I can narrow down results and find my item faster."

Acceptance criteria:

Given	When	Then
Keyword entered	Search executed	Matching reports are displayed.

US.SEARCH.03: Identify item status clearly | ReqRef: [REQ-SEARCH-F-03]

"As a user who lost an item, I want to clearly see whether a listing is still lost or has been found so that I can focus only on relevant reports."

Acceptance criteria:

Given	When	Then
Listings displayed	User views results	Each listing clearly indicates lost or found status

E2 Traceability (Stories → Features → Requirements):

Story ID	Feature	ReqRef	Notes
US-SEARCH-01	F2.1	[REQ-SEARCH-F-01]	Category filtering logic.
US-SEARCH-02	F2.2, F2.3	[REQ-SEARCH-F-02]	Search and sorting.
US-SEARCH-03	F2.4, F2.5	[REQ-SEARCH-F-03]	Report labelling (lost/found).

Epic E3: Safe Claim and Ownership Verification

Goal: Support safe and responsible item recovery.

Problem/value: Returning items without verification may result in disputes or misuse.

Features (F3):

- F3.1 Institutional contact mechanism

- F3.2 Ownership verification guidance
- F3.3 Mark report as resolved
- F3.4 Report suspicious listing

US.CLAIM.01: Contact finder securely | ReqRef: [REQ-CLAIM-F-01]

"As a user who lost an item, I want to contact the finder through institutional channels so that communication remains trusted."

Acceptance criteria:

Given	When	Then
Active report	Contact initiated	Message is sent via institutional email.

US.CLAIM.02: Mark item as resolved | ReqRef: [REQ-CLAIM-F-02]

"As a user who reported a lost item, I want to be able to mark an item as resolved so that others know it has been returned."

Acceptance criteria:

Given	When	Then
Active owned report	Status changed to resolved	Report no longer appears in active search.

US.CLAIM.03: Verify ownership before returning an item | ReqRef: [REQ-CLAIM-F-03]

"As a user who found an item, I want guidance on how to verify ownership so that I can return it to the correct person."

Given	When	Then
User views claim process	Guidance displayed	User sees recommended verification steps

E3 Traceability (Stories → Features → Requirements):

Story ID	Feature	ReqRef	Notes
US-CLAIM-01	F3.1	[REQ-CLAIM-F-01]	Secure communication.
US-CLAIM-02	F3.3	[REQ-CLAIM-F-02]	Lifecycle transition.
US-CLAIM-03	F3.2	[REQ-CLAIM-F-03]	Ownership verification guidance.

Epic E4: Accessibility and Usability

Goal: Ensure the platform is accessible and usable by the university community.

Problem/value: Students and staff use different devices and may have accessibility needs.

Features (F4):

- F4.1 High contrast UI
- F4.2 Text + icon status indicators
- F4.3 Responsive design
- F4.4 Clear visual hierarchy

US.UX.01: Use platform on mobile | ReqRef: [REQ-UX-NF-01]

"As a mobile user, I want the interface to be responsive so that I can use it on campus."

Acceptance criteria:

Given	When	Then
Mobile device	Page loaded	Layout adapts without horizontal scrolling.

US.UX.02: Understand status without relying on color | ReqRef: [REQ-UX-NF-02]

"As a color-blind user, I want status indicators to include labels or icons so that I can understand them properly."

Acceptance criteria:

Given	When	Then
Listings displayed	User views status	Status is understandable without relying on color alone

E4 Traceability (Stories → Features → Requirements):

Story ID	Feature	ReqRef	Notes
US-UX-01	F4.3	[REQ-UX-NF-01]	Responsive design rule.
US-UX-02	F4.2	[REQ-UX-NF-02]	Accessibility for the visually impaired.

Epic E5: Administrative Oversight

Goal: Maintain platform integrity and moderation control.

Problem/value: The system must prevent abuse and ensure trustworthy listings.

Features (F5):

- F5.1 View all reports
- F5.2 Remove inappropriate reports
- F5.3 Role-based access control (RLS enforcement)

US.ADMIN.01: Moderate reports | ReqRef: [REQ-ADMIN-F-01]

"As an administrator, I want to remove inappropriate listings so that the platform remains safe and trustworthy."

Acceptance criteria:

Given	When	Then
Admin privileges	Report marked as removed	Report is hidden from public listings.

E5 Traceability (Stories → Features → Requirements):

Story ID	Feature	ReqRef	Notes
US-ADMIN-01	F5.2, F5.3	[REQ-ADMIN-F-01]	Admin lifecycle control and RLS.

2.2.2. Personas

A persona is a fictional yet plausible representation of the goals, needs, and expectations of any person within the project's domain. These don't just have to be direct users of the platform; anyone who benefits from the project's goals in one way or another can be a good persona. These personas serve to guide development in such a way where users are kept in mind when creating, developing, and refining features.

The following personas represent direct users of the platform:

Name	**Mateo Rivera**
Age	18 years old
Occupation	Freshman computer science student at UPRM
Appearance	Average height, medium build, dark and curly short hair, always wearing a baseball cap and baggy jeans, slightly sleep-deprived look
Hobbies	Basketball, game development, playing video games
Personality	Distracted, tech-savvy, over-achiever
Tech setup	Gaming setup at his dorm with a large monitor, MacBook for schoolwork, and an iPhone 15
Story	Mateo just started university with an 18 credit semester and is still adjusting to the new pace. He moves between classes, the library, and study groups constantly. One afternoon, he realizes he left his laptop charger in a classroom and he has a programming assignment due tomorrow afternoon. He has no spare charger and his laptop is nearly out of battery. His work was saved locally on his laptop and he needs it to complete and submit the assignment.

Name	Mateo Rivera
Day in the life	• 6:30 am : Wakes up and starts getting ready • 7:00 am : Arrives at campus • 7:30 am : Study for a quiz at the university library and review class notes • 8:20 am : Attends class and takes a quiz • 9:30 am : Rushes to second class • 10:30 am : Takes lunch • 12:30 pm : Attends lab • 3:30 pm : Attends last class • 4:45 pm : Goes to a group study section on campus • 6:00 pm : Goes to dorm to take a little nap, and later continues working on a programming a game • 9:00 pm : Works on a project while watching a basketball game • 10:50 pm : Goes to sleep
Goals	Recover lost items as quickly as possible
Pain Points	Doesn't know where to check for lost items, fears items being stolen
Needs	• Fast search functionality • Clear item categorization (electronics, notebooks, etc.) • Timestamp and location information
Platform Interaction	Filters search to look only for lost electronics, accessing the platform from his phone
Keep in mind when	• Designing search filters • Category tags • Mobile responsiveness

Name	Valeria Santiago
Age	21 years old
Occupation	Biology student and lab assistant
Appearance	Short, small build, brunette straight hair with blonde tips, carries a reusable water bottle everywhere
Hobbies	Creating mocktail recipes, jogging, dancing
Personality	Responsible, detail-oriented, empathetic
Tech setup	Uses an older MacBook for schoolwork and an iPhone 17

Name	Valeria Santiago
Story	Valeria occasionally finds lost items in the lab rooms she frequents; some are more important such as car keys and pen drives; others less so, such as water bottles and calculators. Regardless of importance, she feels guilty simply leaving the lost items alone knowing they could be stolen by someone, but there is no lost and found office near her labs.
Day in the life	• 6:30 am: Wakes up and starts getting ready for her daily walk • 8:00 am: Arrives at the University • 8:30 am: Attends her first biology class • 10:00 am : Arrives at the lab and begins preparing and cleaning the workspace • 12:00 pm Finds a calculator on a lab table and asks nearby students if it belongs to anyone but no one claims it. • 12:10 pm : Stores the calculator in a lab drawer and informs the next lab assistant in case someone comes looking for it • 1:00 pm : Eats lunch at a cafe with friends • 2:20 pm : Returns to campus for another class • 4:00 pm: Meets with a friend to study • 5:00 pm: Checks and responds to emails • 6:00 pm: Eats dinner • 7:00 pm: Studies and starts preparing lab material for the next day • 9:00 pm: Watch two episodes of Gilmore Girls • 10:30pm : Goes to sleep
Goals	Quickly report found items so the owner can reclaim them
Pain Points	• Reporting a lost item is inaccessible or, at best, very inconvenient
Needs	• Simple and quick submission form • Ability to upload a photo of the lost item • Minimal required fields
Platform Interaction	Submits reports from her laptop after finishing a lab shift
Keep in mind when	Designing the user flow for reporting a found item

Name	Daniel Torres
Age	24 years old
Occupation	Graduate student and TA (Teaching Assistant) for a physics course

Name	Daniel Torres
Appearance	Large build, tall, short and straight hair, organized, usually wearing a hoodie and black backpack
Hobbies	Sculpture painting, tinkering with cars, reading
Personality	Practical, skeptical, passionate, values privacy
Tech setup	Mid-end desktop in his home for everyday use and an iPhone 12
Story	On his way back to his car after a class, Daniel finds a phone on the ground that clearly fell out of someone's bag. Having heard of the lost and found platform, he decides to submit it to the platform. He gets an institutional email from someone claiming the phone is theirs. He is, however, slightly skeptical of this given that the phone has little to no information that could help identify the person as the rightful owner.
Day in the life	• 5:00 am Wakes up and starts getting ready • 6:30 am : Reviews physics notes and starts preparing for the course • 7:00 am : Stops at a "Panadería" to get coffee and breakfast • 7:50 am : Arrives at campus • 8:30 am: Offers office hours, responds to students emails and grades tests • 10:45 am : Goes to the student center to warm up his lunch and meet with a colleague • 12:30 pm : Takes a class • 1:45 pm : Walks to the parking lot to his car and drives back home • 2:00 pm : Stops for gas • 2:26 pm : Stops at his apartment and play with his cats • 3:00 pm : Visits his grandparents house and has dinner there • 4:45 pm : Goes back home and joins a departamental online meeting • 5:50 pm : Review the course presentation, fix some typos and review the material • 6:55 pm: Corrects some tests • 8:35 pm : Takes a shower and reads his favorite book • 12: 00 am : Goes to sleep
Goals	Ensure the lost item is being returned to its true owner
Pain Points	• Difficulty verifying the legitimacy of ownership claims • Fear of giving a lost item to the wrong person • Lack of guidance on how to confirm ownership safely • Concern about exposing his own personal contact information

Name	**Daniel Torres**
Needs	• Institutional email restriction • A way to inform of scammers/impersonators • Clear ownership verification guidelines (e.g., ask claimant to describe lock screen, phone case, distinguishing features)
Platform Interaction	Submits the found item through his desktop later that evening and communicates via institutional email, carefully reviewing responses before arranging a handoff
Keep in mind when	Defining authentication requirements, claim verification processes, and privacy constraints

Name	**Luis Iglesias**
Age	34 years old
Occupation	Graduate student doing a masters degree in Finance
Appearance	Average height, fit build, neatly trimmed beard, usually dressed in business-casual attire with a leather messenger bag
Hobbies	Politics, running, watching basketball, budgeting and personal finance planning
Personality	Analytical, calm, detail-oriented
Tech setup	Mid-end laptop used for classwork and more serious affairs and Google Pixel phone
Story	Luis has mild color vision deficiency (red-green color blindness) and is sensitive to overly bright or low-contrast interfaces. He frequently uses his phone outdoors where glare makes low-contrast text difficult to read. This causes him issues with not being able to comfortably use platforms where color contrast isn’t properly considered. When Luis loses his keys around campus, he decides to use the lost and found platform and finds he is able to navigate it just fine despite his vision deficiency.

Name	Luis Iglesias
Day in the life	• 8:30 am : Wakes up and drinks a cup of coffee • 9:00 am : Uses his computer and starts working on his online job as finance counselor • 1:00 pm : Eats lunch • 2:20 pm : Arrives at college to attend class • 3:50 pm : Goes to Gym • 4:45 pm : Goes to the Supermarket do buy food • 6:23 pm : Cooks dinner • 7:30 pm : Watches a baseball game • 8:00 pm : Uses his phone to watch social media • 9:48 pm : Prepares the clothes for the next day • 10:47 pm : Goes to sleep • 12: 05 am : Wakes up and drink a cup of water • 12:25 am : Reviews emails and text messages • 1:08 am : Goes back to sleep
Goals	Read and navigate through content easily despite visual impairment
Pain Points	• Low contrast between text and background • Status indicators shown only through color (e.g., “red = lost”, “green = found”)
Needs	• High-contrast, accessible color palette • Clear typography hierarchy • Labels and icons in addition to color indicators
Platform Interaction	Uses his phone outdoors and his laptop at night
Keep in mind when	Choosing the color palette and designing UI elements

As mentioned, it’s not just direct users of the platform that benefit from it; other people within the domain can also benefit from the platform’s goals. The following persona pertains to the project’s domain:

Name	Camila López
Age	19 years old
Occupation	Student working at the university’s library part-time
Appearance	Tall, average build, short and wavy hair in a pixie cut, often seen wearing a sweater

Name	Camila López
Hobbies	Sewing plushies, illustrative work, reading
Personality	Shy, humble, warm personality
Tech setup	Low-end laptop used almost exclusively for schoolwork and a Samsung Galaxy phone
Story	Camila works a part-time job at the university's library on the third floor office where study room reservations and lost items are handled. Students frequently bring in found items to the security office such as water bottles, ID cards, keychains, and even electronics. Managing these items manually creates clutter and confusion. Students often ask her if someone reported a lost item, but she has no proper way to keep track. Camila does not personally use the platform daily, but its existence directly impacts her workflow by reducing the number of items she has to manage.
Day in the life	• 10:00 am : Wakes up and starts getting ready for college • 11:00 am : Arrives at the University and eats at the cafeteria • 11:30 am : Reports at the Library to her job • 3:20 pm : Heads to her class • 4:34 pm : Walks to her dorm while listening to a audio book • 5:00 pm : Finish a crochet plushie that she has to deliver the next day • 6:46 pm : Discuss with her roomate the ending of a book she finished the night before • 7:49 pm : Study mandarin using Duolingo • 8:50 pm : Call her parents and clean the dorm • 9:50 : Goes to sleep
Goals	• Reduce physical storage of lost items • Avoid disputes over ownership • Improve campus trust and accountability
Pain Points	• Manual logging of items • Students arguing over ownership • Lack of structure
Needs	• Reduced reliance on the library's third-floor office as the "default" lost & found • A system that minimizes false claims
Platform Interaction	Indirect stakeholder affected by how students use the system
Keep in mind when	Defining ownership verification rules and dispute scenarios

Name	**Gaston Carlo**
Age	22 years old
Occupation	Fifth year student Nursing student
Appearance	Tall, natural red hair, freckles. Usually, uses Marvel superheroes t-shirts.Always has his smart watch.
Hobbies	Reading comics, watching superheroes movies and Grey's Anatomy, Hiking
Personality	Considerate, attentive, helper, outgoing
Tech setup	Smart watch, Iphone 15 and a HP laptop used mainly for assignments and watching streaming services
Story	Gaston found a sweater on the Nursing auditorium after class. He asked around if it belonged to anyone, but nobody claimed it, since he was in a hurry and didn't know of any official reporting system, he left the sweater folded in a corner of the room, assuming the owner would come back for it. One month later, a classmate accidentally spilled water on herself and was visibly uncomfortable since the classroom was very cold. Gaston saw the sweater in a corner and offered it to her
Day in the life	• 8:30 am : Wakes up and looks at social media apps • 9:00 am : Prepares backpack and starts getting ready for class • 9:35 am : Picks up a friend for college • 9:46 am : Arrives at the University an is late for class • 9:50 am : Rushes to the Nursing auditorium, notice items left behind from previous class, but ignores it due to time pressure • 11:30 am : Reunites with a proffessor to talk about his nursery practices • 12:30 pm : Attends lab • 3:30 pm : Eats a snack while finishing a homework paper • 4:45 pm : Attends class • 6:00 pm : Goes to the Mayaguez Mall to buy new shoes for his practice at the hospital • 9:00 pm : Prepares his clothes and hospital backpack with all the essentials • 10:50 pm : Reviews class notes to make sure he performs basic vital signs properly and doesn't forget anything • 12:00 am : Goes to sleep
Goals	Help others in immediate situations
Pain Points	• Doesn't see value in time consuming reporting processes • Lack of awareness of existing reporting systems
Needs	• Very quick and easy reporting system • Awareness mechanisms(e.g. reminders about reporting tools)

Name	**Gaston Carlo**
Platform Interaction	Rarely interacts with the platform. Would only use it if the reporting process is fast and not overcomplicated
Keep in mind when	• Designing a report template that is not overcomplicated or time comsuming • Creating awareness of the platform around campus • Designing for users with low motivation to report items

Name	**Sara Lopez**
Age	47 years old
Occupation	Works at a travel agency
Appearance	Short, curly ombre long hair. Dresses stylishly, always carries a big purse
Hobbies	Traveling, cooking,organizing family events
Personality	Caring, attentive, responsible,practical, time-conscious
Tech setup	Samsung s26, smartwatch, desktop computer mainly used for work and a personal computer also used for work and personal tasks
Story	Sara visits the university campus often to pick up or drop off her son. One afternoon, while waiting for her son she decides to take a walk on campus. While walking she notices a backpack and a tablet left on a bench.She waits to see if the owner comes back, but after 10 minutes no one came for the items.She heard about the University platform to report lost items and considers reporting the item but realizes she does not have a university account to access the platform.She picks up the items and leaves them at a nearby office. She asks her son to report the items on the platform, so the owner knows they found the items. She hopes the owner sees the report and retrives the items

Name	**Sara Lopez**
Day in the life	• 4:30 am : Wakes up, prepares breakfast, cleans the kitchen and the living room • 5:35 am : Checks her schedule to see all upcoming appointments of the day • 6:00 am : Wakes her son up and tells him to prepare for college • 7:00 am : Drop her son and drives to work • 12:30 am : Eats lunch, while taking with collegues about new travel offers • 3:30 pm : Drives to college to pick up her son • 4:00 pm : Walks on campus while waiting that his son finishes his last class • 4:10 pm : Notices unattended backpack and tablet on a bench and waits to see if someone comes to pick up the items • 4:20 pm : After waiting, considers what to do but is unsure how to report them • 4:25 pm : Reunites with her son and leaves the items at a nearby office • 4:50 pm : Is worried about the items owner and hopes that the person can retrieve its lost items, so she asks her son to create a report, to let know the owner that they found the items • 5:00 pm: Goes home and prepares dinner • 6:00 pm: Plays boardgames with the family • 7:34 pm: Watch an episode of her favorite show • 9:00 pm : Goes to sleep
Goals	Handle situations quickly and responsibly without disrupting her schedule
Pain Points	• Doesn't see value in time consuming reporting processes • Lack of awareness of existing reporting systems
Needs	• Guest or public reporting option without requiring a university account • Fast and simple reporting process with minimal steps • Clear guidance on what to do when finding lost items
Platform Interaction	Currently unable to report on the platform
Keep in mind when	• Designing access for non-university users (e.g. parents, visitors) • Ensuring the system is intuitive for people unfamiliar with reporting processes • Providing quick and accessible reporting alternatives for time-constrained users

Name	**Krumi Silva**
Age	21 years old

Name	**Krumi Silva**
Occupation	Junior mechanical engineering student
Appearance	Average height, short dark hair, usually wearing t-shirts and flip flops
Hobbies	Gaming, skateboarding, restoring damaged tech items
Personality	A little selfish, slightly opportunistic
Tech setup	Iphone 16, personal computer for classwork
Story	Krumi finds a power bank left on a library table while he was studying, he notices no one is around. Several hours passed and no one claims it. He debates whether to report it but thinks, "It's been here for a while and I could really use it". He pockets it, telling himself he'll return it later in case someone reports it as a lost item. Days go by, and he decides to keep it, rationalizing that no one has claimed and it's helping him manage his busy schedule
Day in the life	• 10:00 am : Wakes up,grabs breakfast • 10:25 am : Walks to the library for morning study section • 10:54 am : Notices a power bank on the table, no one is around, hesitate to report it or leaving it • 12:45 pm : Decides to take it with him temporarily • 1:20 pm : Attends class, keep power bank on his backpack • 2:30 pm : Eats lunch in the Cafeteria, uses the power bank to charge his phone • 3:30 pm : Attends second class • 4:00 pm : Meets with friends, tells them about the power bank he found. Friends tell him, that if no one has claimed it, he should keep it • 5:10 pm : Goes home, study for a test, charges the power bank • 6:00 pm : Plays video games all night • 1:23 am : Goes to sleep
Goals	Resolve situations quickly while minimizing personal effort, even if that involves keeping found items
Pain Points	• Not visible or well-known lost and found system on campus • Ethical uncertainty • Inconvenience of reporting lost items
Needs	• Simple remainders for ethical reporting • Clear, accessible lost and found location points, to reduce confusion and encourage the returning of found items • Awareness of reporting processes
Platform Interaction	Rarely interacts

Name	**Krumi Silva**
Keep in mind when	• Designing edge cases of users who may keep items • Remainders about reporting found items • Considering ethical reminders • Understanding motivations for retaining items rather than reporting them

Key Insights derived from Personas:

- Not all users are motivated to report found items, therefore the system must minimize effort and improve accessibility.
- Some users may justify keeping found items(will never report them as found)
- Restricting access to university members excludes the participation of external possible contributors such as parents, guests, visitors, community members.This suggest the need of a guest reporting option.
- Users may fear giving the found items to the wrong person requiring clear ownership verification
- Accessibility is critical for usability

2.2.3. Domain Requirements

This section defines functional requirements that arise directly from the lost-and-found domain described in the narrative. Each requirement is grounded in domain properties such as delayed reporting, distributed custody, ownership uncertainty, and the need for structured matching and verification.

Reporting Requirements

Required Report Content

Domain basis: Items are often non-unique (e.g., chargers, keys), and false matches occur when descriptions are insufficient.

Decision: Structured reports with sufficient descriptive information are required to support reliable matching.

Requirements:

- REQ-REPORT-F-01: The system must accept only **Reports** that include sufficient descriptive attributes to support **Matching**, including at minimum:
    - item category
    - textual description
    - approximate location (loss or discovery)
    - relevant date or time window
- REQ-REPORT-F-02: The system must separate descriptive information into:

- public attributes (used for search and matching)
- private attributes (used for **Claim Verification**)

Report Initialization

Domain basis: All reports represent unresolved situations at the moment of creation.

Requirements:

- REQ-REPORT-F-03: The system must assign the state **Active** to every newly created **Report**.

Finder Traceability

Domain basis: The person who finds an item is not always the same as the one who returns it, and follow-up communication may be required.

Requirements:

- REQ-REPORT-F-04: The system must record the **Finder** associated with a **Found Item Report**.

Report Ownership

Domain basis: Reports originate from identifiable domain actors (loser or finder), and ownership determines who can modify or verify report information.

Requirements:

- REQ-REPORT-F-05: The system must associate each **Report** with its creator to support ownership, traceability, and controlled modification.
- REQ-REPORT-F-06: The system must require a registered UPRM community member to create a **Report** for a **Lost Item**.
- REQ-REPORT-F-07: The system must require a registered UPRM community member to create a **Report** for a **Found Item**.

Search and Discovery Requirements

Search Mechanism

Domain basis: Items lack unique identifiers and are described using non-standard, free-form language, requiring retrieval based on partial descriptions.

Requirements:

- REQ-SEARCH-F-01: The system must support retrieval of **Reports** based on descriptive attributes of items, including:
  - textual description
  - item category
  - approximate location
  - date or time window

- REQ-SEARCH-F-02: The system must support filtering of **Reports** by:
    - category
    - location
    - date
    - report state
- REQ-SEARCH-F-03: The system must present **Reports** in a form that enables comparison between **Lost Item Reports** and **Found Item Reports** to support **Matching**.

Matching and Tracking Requirements

Domain Basis

Matching is required because lost and found reports are created independently and must be correlated based on partial information.

Requirements:

- REQ-MATCH-F-01: The system must support comparison of **Lost Item Reports** and **Found Item Reports** based on descriptive attributes to identify potential matches.
- REQ-MATCH-F-02: The system must maintain the current state of each **Report**, including:
    - Active
    - Returned
    - Closed
    - Expired
- REQ-MATCH-F-03: The system must record transitions between report states.

Owner Tracking

Domain basis: The individual who lost an item needs visibility into the recovery process.

Requirements:

- REQ-MATCH-F-04: The system must provide visibility of the current state of a **Report** to its associated owner.

Claim Verification and Recovery Requirements

Ownership Verification

Domain basis: False claims may occur due to non-unique items and public visibility of reports.

Requirements:

- REQ-CLAIM-F-01: The system must restrict confirmation of **Item Returned** to authenticated actors associated with the report.
- REQ-CLAIM-F-02: The system must require validation of ownership using private report

attributes before confirming **Item Returned**.

Recovery Definition

Clarification:

- **Recovery** corresponds to the event **Item Returned**
- **Returned** is a terminal state representing successful closure
- **Closure** may occur as:
- successful closure (**Returned**)
- manual closure (**Closed**)
- time-based closure (**Expired**)

Requirements:

- REQ-CLAIM-F-03: The system must transition a **Report** from **Active** to **Returned** upon successful recovery.

Closure Requirements

Non-Recovery Closure

Domain basis: Not all items are recovered; reports may need to be terminated without recovery.

Requirements:

- REQ-CLOSE-F-01: The system must support manual transition of a **Report** from **Active** to **Closed** when recovery is no longer expected.
- REQ-CLOSE-F-02: The system must support automatic transition of a **Report** from **Active** to **Expired** after a defined period of inactivity.
- REQ-CLOSE-F-03: The system must treat **Returned**, **Closed**, and **Expired** as terminal states.
- REQ-CLOSE-F-04: The system must retain all non-active reports for reference.

Visibility of Closed Reports

Domain basis: Inactive reports must not interfere with ongoing recovery efforts but remain accessible for reference.

Requirements:

- REQ-CLOSE-F-05: The system must distinguish non-active reports from **Active** reports in all retrieval and search results.

Administrative Oversight Requirements

Domain basis: Reports may contain incorrect, misleading, or inappropriate content and require institutional oversight.

Requirements:

- REQ-ADMIN-F-01: The system must support administrative review of **Reports**.
- REQ-ADMIN-F-02: The system must support modification and removal of **Reports** by authorized administrators when they violate institutional policies.
- REQ-ADMIN-F-03: The system must support administrative override of report states in cases of dispute or misuse.

Accessibility and Usability Requirements

Domain basis: Users interact with the system in diverse environments and devices.

Requirements:

- REQ-UX-NF-01: The system must provide a responsive, high-contrast interface that adapts to mobile, tablet, and desktop environments without loss of usability.

2.2.4. Reporting Interface Requirements

ID	Interface Requirement	Domain Phenomenon	Linked User Stories/Epics
IR001	When an Owner reports a lost item, the system creates a Lost Item Report representing the real-world loss event. It initializes all descriptive attributes (category, description, location, time) using the Owner's observation data, and any subsequent updates overwrite previous values while preserving historical entries.	The real-world phenomenon is the loss of an item by an Owner; internally represented as a Lost Item Report; updates occur whenever the Owner modifies the report, keeping historical versions.	• [US-REPORT-01]
IR002	When a Finder reports a found item, the system generates a Found Item Report linked to the Finder. The report's descriptive attributes reflect the initial observation and are replaced if the Finder edits the information, keeping a record of previous entries.	Discovery of an item by a Finder; internal representation as a Found Item Report; updates occur when the Finder edits attributes, preserving history.	• [US-REPORT-02]
IR003	The system records category, description, location, and time directly from Owner or Finder input and updates the internal representation immediately whenever these attributes change.	Item characteristics observed by domain actors (Owner/Finder); internal representation mirrors input; modifications immediately update the internal record.	• [US-REPORT-01] • [US-REPORT-02]

ID	Interface Requirement	Domain Phenomenon	Linked User Stories/Epics
IR004	The system distinguishes public and private attributes. Public attributes display to all domain actors, while private attributes store only Owner-provided knowledge and restrict access to the Owner.	Ownership-specific knowledge; internal representation enforces public/private access; updates occur only from the Owner for private data.	• [US-CLAIM-02]
IR005	Owners or Finders can correct or refine descriptive attributes; the system immediately replaces previous values and records modifications in the report's history.	Evolving or incomplete knowledge; internal attributes update with new domain actor input while historical records maintain previous values.	• [US-REPORT-01] • [US-REPORT-02]
IR006	The system locks immutable attributes (report type, creation timestamp) and rejects any attempted modifications, generating an error notification.	Immutable domain events (report type, creation time) are represented as unmodifiable fields; attempted changes are blocked and logged.	• [US-REPORT-01] • [US-REPORT-02]
IR007	Each report maintains a state attribute reflecting the item's real-world lifecycle. The system updates the state automatically according to domain events and confirmations from Owners or Finders.	Item lifecycle state in the domain; internal state updates mirror real-world transitions triggered by domain actor actions or system rules.	• [US-CLAIM-02]
IR008	When an Owner confirms an item has been returned, the system updates the report state to Returned and records the timestamp and Owner identity.	Recovery event by Owner; internal state set to Returned; updates occur upon Owner confirmation.	• [US-CLAIM-02]
IR009	The system enforces authentication for recovery confirmations, associates the confirming Owner with the report, and logs the event in the report's history.	Trusted ownership confirmation; internal representation links authenticated Owner to the report and logs confirmation; updates only on valid authentication.	• [US-CLAIM-01] • [US-CLAIM-02]

ID	Interface Requirement	Domain Phenomenon	Linked User Stories/Epics
IR010	Administrators can manually transition a report to Closed; the internal state updates and records the timestamp and Administrator identity, preventing further modifications.	Abandonment of recovery effort; internal state updates to Closed; timestamp and Administrator identity recorded; further modifications blocked.	• [US-ADMIN-01]
IR011	Reports automatically transition to Expired when the system detects inactivity beyond the defined period, updating the internal state and recording the timestamp.	Passage of time without activity; internal state changes to Expired automatically; timestamps logged.	• [US-ADMIN-01]
IR012	The system records every state transition with timestamps and prevents deletion or alteration of historical state entries.	Temporal evolution of report state; internal representation stores all state changes immutably; updates reflect domain changes while preserving history.	• [US-CLAIM-02]
IR013	All reports persist after reaching terminal states (Returned, Closed, Expired), allowing retrieval for audits or references without loss of information.	Persistence of past events; terminal states prevent deletion; reports retrievable for reference.	• [US-ADMIN-01] • [US-CLAIM-02]
IR014	The system exposes internal report attributes to enable Owners or Finders to compare items with real-world observations and determine matches, updating search results dynamically.	Recognition of items by domain actors; internal searchable attributes; updates occur as observations change.	• [US-SEARCH-01] • [US-SEARCH-02]
IR015	The system filters reports by category, location, date, and state based on criteria selected by domain actors, updating query results immediately.	Human filtering behavior by Owners/Finders; internal representation mirrors selection and updates results dynamically.	• [US-SEARCH-01] • [US-SEARCH-02]

ID	Interface Requirement	Domain Phenomenon	Linked User Stories/Epics
IR016	The system compares Lost Item Reports with Found Item Reports by evaluating shared descriptive attributes, presenting the comparison to domain actors and updating internal match states.	Matching across independent reports; internal comparison state updates when attributes change; results presented to domain actors.	• [US-SEARCH-02]
IR017	The system mediates communication between Owners and Finders regarding a report, enforcing separation of public and private information and logging all interactions internally.	Communication between domain actors; internal logs and access controls; updates occur with each interaction.	• [US-CLAIM-01]
IR018	Administrators modify or remove report representations directly, with all changes logged in the audit trail and internal state updated immediately.	Administrative oversight; internal state reflects modifications/removals; audit logs preserve event history.	• [US-ADMIN-01]

2.2.5. Machine Requirements

Table 1. Hardware, Software, and Environmental Requirements

ID	Description	Linked User Stories/Epics	Verification Tests
HWRQ-001	The system must support client access from desktop/laptop devices with modern web browsers and minimum 1 Mbps internet connection.	US-Platform-001: Web-Based Access, US-Perf-002: Performance Standards	T-PERF-005, T-COMP-010
HWRQ-002	The system must support client access from mobile devices (smartphones/tablets) with responsive design at minimum 320px screen width.	US-Platform-001: Web-Based Access, US-UI-003: Mobile Responsiveness	T-UI-015, T-RESP-001
SWRQ-001	The frontend must be developed using TypeScript 5.0+ and React 18.2+ with Node.js 18.0+ runtime.	US-Auth-001: Authentication System, US-Report-001: Report Creation	T-COMP-001, T-BUILD-001

ID	Description	Linked User Stories/Epics	Verification Tests
SWRQ-002	The system must integrate Supabase JavaScript SDK 2.0+ for backend services including authentication, database, and storage.	US-Auth-001: Authentication System, US-Report-002: Database Storage	T-API-001, T-DB-001
SWRQ-003	The system must utilize TailwindCSS 3.0+ for responsive UI implementation.	US-UI-001: User Interface Design, US-UI-003: Mobile Responsiveness	T-UI-010, T-UI-011
SWRQ-004	The system must implement form validation using React Hook Form with Zod or Yup schema validation.	US-Report-003: Form Validation, US-Report-004: Data Integrity	T-FE-005, T-VALID-001
SWRQ-005	The system must support image upload functionality with file size limit of 10MB per image for JPEG, PNG, and GIF formats.	US-Report-005: Image Upload, US-Sec-003: File Security	T-UPLOAD-001, T-SEC-008
SWRQ-006	The development environment must include Jest, React Testing Library, and Cypress/Playwright for testing.	US-QA-001: Testing Requirements, US-QA-002: Quality Assurance	T-TEST-001, T-TEST-002
ENVRQ-001	The system must maintain API response latency under 500ms for standard operations under normal load.	US-Perf-001: System Performance, US-Perf-002: Performance Standards	T-PERF-001, T-PERF-002
ENVRQ-002	The system must support concurrent users with minimum 100 simultaneous active sessions.	US-Perf-003: Scalability, US-Perf-004: Load Handling	T-PERF-010, T-SCALE-001
ENVRQ-003	The system must enforce HTTPS-only communication with WSS for real-time features.	US-Sec-001: Data Security, US-Sec-002: Secure Communication	T-SEC-001, T-SEC-002
ENVRQ-004	The system must support latest 2 versions of Chrome, Firefox, Safari, and Edge browsers on desktop and mobile.	US-COMP-001: Browser Compatibility, US-UI-003: Mobile Responsiveness	T-COMP-005, T-COMP-006

ID	Description	Linked User Stories/Epics	Verification Tests
ENVRQ-005	The system must integrate with UPR.edu authentication system via SSO/OAuth protocols.	US-Auth-002: UPR Authentication, US-Auth-003: SSO Integration	T-AUTH-010, T-AUTH-011
ENVRQ-006	The system must implement Row Level Security (RLS) policies in PostgreSQL for all data access.	US-Sec-004: Access Control, US-Sec-005: Data Privacy	T-SEC-015, T-DB-010
ENVRQ-007	The system must achieve page load time under 3 seconds and search query responses under 2 seconds.	US-Perf-001: System Performance, US-Search-003: Search Optimization	T-PERF-003, T-PERF-004
ENVRQ-008	The system must comply with FERPA regulations and university data protection policies for student information.	US-COMP-002: Regulatory Compliance, US-Sec-006: Privacy Compliance	T-COMP-020, T-SEC-020
ENVRQ-009	The system must implement rate limiting on all API endpoints to prevent abuse.	US-Sec-007: Security Hardening, US-Sec-008: Abuse Prevention	T-SEC-025, T-SEC-026
ENVRQ-010	The system must support email notifications through configured SMTP with templated messaging.	US-Notif-001: Email Notifications, US-Comm-003: System Communications	T-NOTIF-001, T-EMAIL-001

2.3. Implementation

2.3.1. Selected Fragments of the Implementation

NOTE The following fragments are extracted from the current implementation using React (TypeScript) and Supabase. They are included only where natural language alone would make architectural or behavioral explanations unnecessarily abstract. Each fragment directly supports a design decision explained in the accompanying text.

Authentication via UPR SSO (Supabase OAuth)

The system relies on institutional authentication. Rather than managing passwords internally, authentication is delegated to the university’s SSO provider through Supabase OAuth integration.

This supports three domain requirements:

- Institutional identity verification

- No password storage within the application
- Stateless session validation using JWT

The following fragment shows the login initiation from the React client:

```
import { createClient } from "@supabase/supabase-js";

const supabase = createClient(
  import.meta.env.VITE_SUPABASE_URL!,
  import.meta.env.VITE_SUPABASE_ANON_KEY!
);

export async function signInWithUPR() {
  const { error } = await supabase.auth.signInWithOAuth({
    provider: "azure", // UPR SSO provider
    options: {
      redirectTo: `${window.location.origin}/auth/callback`
    }
  });

  if (error) {
    throw new Error(error.message);
  }
}
```

This fragment clarifies that:

- Authentication is fully delegated to the identity provider.
- The frontend never handles raw credentials.
- Supabase manages token issuance and session persistence.

Report Creation Flow (Lost / Found)

The system follows a **create-then-persist** workflow: first an in-memory report instance is constructed, then it is saved to the database. This separation clarifies the difference between a draft report (without system-assigned fields) and a stored record.

When a user reports a lost or found item, the system must:

1. Validate structured input.
2. Persist the record.
3. Associate it with the authenticated user.
4. Store images separately.

The following type hierarchy makes the lifecycle explicit:

```
// Domain types
```

```
type ReportType = "lost" | "found";
type UUID = string;

// Abstract superclass for all reports
abstract class Report {
  constructor(
    public readonly type: ReportType,
    public readonly title: string,
    public readonly description: string,
    public readonly categoryId: UUID,
    public readonly location: string,
    public readonly eventDate: Date, // not just a string
    public readonly imageUrl?: URL // not just a string
  ) {}
}

// Input representation (before persistence)
class ReportInput extends Report {}

// Persisted representation (after persistence)
class ReportPersisted extends Report {
  constructor(
    type: ReportType,
    title: string,
    description: string,
    categoryId: UUID,
    location: string,
    eventDate: Date,
    imageUrl: URL | undefined,
    public readonly id: UUID,
    public readonly userId: UUID,
    public readonly status: "active" | "resolved" | "expired",
    public readonly createdAt: Date
  ) {
    super(type, title, description, categoryId, location, eventDate, imageUrl);
  }
}
```

The insertion function now follows a clear **create-then-persist** pattern: an input instance is created first, then its data is mapped to the database schema.

```
export async function createReport(input: ReportInput): Promise<ReportPersisted> {
  // Authentication check
  const {
    data: { user },
    error: authError
  } = await supabase.auth.getUser();

  if (authError || !user) {
    throw new Error("User not authenticated");
```

```
  }

  // Prepare database record from the input instance
  const dbRecord = {
    type: input.type,
    title: input.title,
    description: input.description,
    category_id: input.categoryId,
    location: input.location,
    event_date: input.eventDate.toISOString(), // serialize Date
    image_url: input.imageUrl?.toString(),
    user_id: user.id,
    status: "active"
  };

  // Persist
  const { data, error } = await supabase
    .from("reports")
    .insert([ dbRecord ])
    .select()
    .single();

  if (error) {
    throw error;
  }

  // Rehydrate a ReportPersisted instance
  return new ReportPersisted(
    data.type,
    data.title,
    data.description,
    data.category_id,
    data.location,
    new Date(data.event_date),
    data.image_url ? new URL(data.image_url) : undefined,
    data.id,
    data.user_id,
    data.status,
    new Date(data.created_at)
  );
}
```

These fragments demonstrate several architectural decisions:

- Authentication context is retrieved at runtime.
- Ownership is enforced at insertion (`user_id` bound to session).
- Status lifecycle begins as `"active"`.
- Supabase Row Level Security (RLS) policies enforce access constraints.
- The class hierarchy separates **input** (no system fields) from **persisted** (with id, status,

timestamps), making the create-then-persist workflow explicit.

- Precise types (`Date`, `URL`, `UUID`) replace generic strings, improving maintainability and domain alignment.

Without this code, it would be difficult to concretely show how domain concepts such as **Reporting**, **Ownership**, and **Lifecycle State** are realized.

The diagram below illustrates the flow of data and control during report creation, highlighting the interaction between the React client and Supabase backend, as well as the enforcement of ownership and security policies:

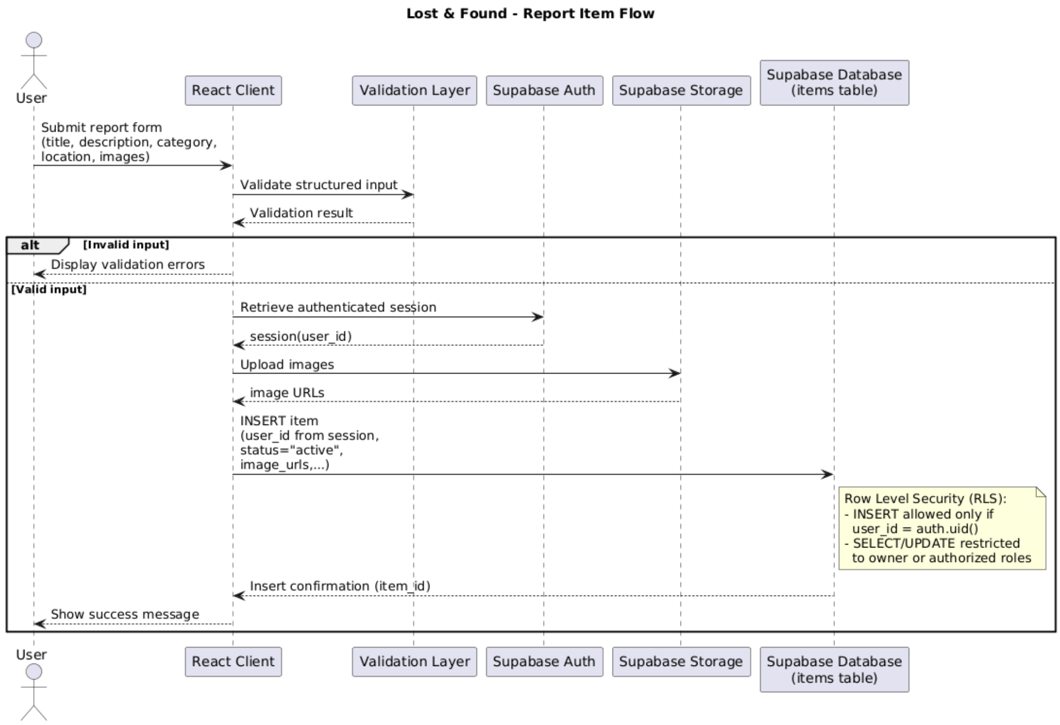

Figure 2.3.1. - 1. Report Creation Flow (React + Supabase).

As shown, the user initiates the report creation from the React client, which then interacts with Supabase to authenticate the user and insert the report into the database. The ownership of the report is established through the `user_id` field, and RLS policies ensure that only the owner can modify their reports.

Row-Level Security (Ownership Enforcement)

The domain requires that users can only edit their own reports. This is not enforced in the React client alone, but at the database layer through RLS.

The following SQL policy fragment is deployed in Supabase:

```
CREATE POLICY "Users can update their own reports"
ON reports
FOR UPDATE
USING (auth.uid() = user_id);
```

This fragment clarifies:

- Authorization is enforced at the database level.
- Client-side manipulation cannot bypass ownership rules.
- Security logic is centralized and declarative.

The policy operationalizes the abstract concept of **Claim Verification** and **Ownership Integrity** in the domain model.

Image Upload Handling

Images are optional but important for identification. Rather than storing binary data inside the reports table, images are stored in Supabase Storage and referenced via URL.

```
export async function uploadImage(file: File): Promise<URL> {
  const filePath = `reports/${crypto.randomUUID()}-${file.name}`;

  const { error } = await supabase.storage
    .from("report-images")
    .upload(filePath, file);

  if (error) throw error;

  const { data } = supabase.storage
    .from("report-images")
    .getPublicUrl(filePath);

  return new URL(data.publicUrl);
}
```

This fragment illustrates:

- Separation of structured metadata (PostgreSQL) and binary assets (Storage).
- Use of UUID-based naming to avoid collisions.
- Public URL generation without exposing storage internals.
- Explicit `URL` return type, reinforcing that the result is a well-formed URL, not an arbitrary string.

This design directly supports the domain behavior of **Matching** by enabling visual comparison while maintaining structural database normalization.

Chapter 3. Analytic Part

3.1. Concept Analysis

This section refines the raw material from the descriptive part into a structured set of domain concepts, abstractions, and relationships. The process began by reviewing unprocessed notes such as “students post lost items,” “admins verify suspicious reports,” “images must be uploaded,” and “users should only edit their own reports.” Recurring patterns were abstracted into domain-level concepts that clarify structure and responsibility.

The result is a shared vocabulary that supports requirements engineering, architecture decisions (React + Supabase), and future evolution of the system.

3.1.1. Method

We systematically reviewed raw descriptions from Section 2, including:

- Informal interview fragments (e.g., “I lost my AirPods in the library,” “People post fake found items”).
- Brainstormed terminology (lost item, found item, claimant, verification, report status).
- Early UI sketches (form submission, image upload, dashboard view).
- Functional expectations (authentication required, ownership enforcement, admin moderation).

Each recurring entity, event, or behavior was evaluated to determine whether it represents:

- A persistent domain entity,
- A domain event,
- A lifecycle state,
- A system behavior or policy.

Ambiguous terms were clarified and generalized where possible to ensure consistency across the team.

3.1.2. Key Concepts Identified

Item: Any physical object that can be lost or found.

Report: [.changed]# A representation of an occurrence in which a person communicates that an item was either lost or found. This communication may take different forms, such as a note, a bulletin board post or a digital entry. It captures essential information about the event such as title, description, category, location, timestamp, and images.#

The report is distinct from the item itself; the item exists in the real world, while the report is its system representation.

People roles: **User**: Interview notes distinguished between regular users and administrators. We generalized this into the concept of a **User**, representing any authenticated actor in the system.uprm-inso4115-2025-2026-s2/semester-project-lost-and-found

Finder: person who found the item. **Owner**: person who lost the item. **Holder**: person responsible for keeping found items.

The owner, finder and holder could be students, community member

Subtypes include:

* Regular user (student/community member),

* Administrator: moderation authority.

Ownership: Relationship between a report and the person who created it. It identifies who is responsible for the report on the domain.[.removed]Ownership is enforced at insertion time (user_id bound to session) and protected through Supabase Row Level Security (RLS).

Image Asset: Raw notes frequently mentioned uploading photos separately from form data. This led to the abstraction of an **Image Asset** as a distinct concept stored in object storage.

Images are associated with reports but managed independently at the infrastructure level.

Report Type: Distinguishes reports as either “lost” or “found”. This distinction reflects important semantic differences in the domain:

- **Lost Item report**: typically submitted by someone who lost the item
- **Found Item report**: typically submitted by someone who found the item

Although both report types share common description elements(such as title, description, category), they differ in the type and reliability of available information. A finder typically knows where the item was found and can often provide images, while an owner may not know the exact location or be able to provide visual evidence.

Despite the differences, both are modeled under a single Report structure to mantain conceptual unity, while allowing certain attributes such as location or images to be optional. This preserves structural simplicity while maintaining semantic clarity.

Status / Lifecycle State: Notes such as “mark as resolved,” “archive old posts,” and “remove inappropriate reports” led to the abstraction of a **Lifecycle State**.

Reports evolve over time. A report begins as active and may transition to states such as:

- `resolved`
- `archived`
- `removed`

This clarifies the changes in the outcome of the reported occurrence.

Claim / Recovery Event: Some narratives included situations such as "I contacted the person and got my item back." This was abstracted as a **Recovery Event**, representing the successsful return of an item through coordination between the involved parties (e.g. finder and owner) It reflects the resolution of the occurrence associated with a report.

Moderation: Concerns about spam or misleading reports were generalized into the concept of **Moderation**, a privileged capability assigned to administrators. Moderation affects report lifecycle and visibility.

3.1.3. Clarifications and Resolutions

- “Lost item” and “Found item” are not separate entities; both are modeled as **Report** instances with different types.
- “Post” and “Report” are treated synonymously, but “Report” is adopted as the formal domain term.
- “Delete” is distinguished from “Archive.” Deletion represents removing a report from consideration(no longer active or visible). Permanent deletion is not essential in this domain. Archiving retains it for reference while marking it as no longer active. This preserves historical information and avoids losing potentially relevant data.

These clarifications prevent semantic confusion in later stages of analysis and develoment.

3.1.4. Relationships and Abstractions:

- A **Person** can create one or more **Reports**.
- [.changed] Each **Report** is associated with exactly one **Person**
- A **Report** may have multiple **Image Assets**.
- A **Report** has one **Report Type** (`lost` or `found`).
- A **Report** has one **Lifecycle State** at any time.
- An **Administrator** can modify lifecycle states of any report.
- A **Recovery Event** transitions a report from `"active"` to `"resolved"`.

These relationships formalize previously informal interactions.

3.1.5. Reasoning and Process (Domain Level)

This concept analysis is grounded in observations from the descriptive part such as narratives of lost and found items:

- Recurring mentions of reporting occurences and coordinating recovery, led to formalizing **Reports**,**Image Assets** and **Claim/Recovery Event**
- Concerns about inappropriate or outdated posts led to modeling **Lifecycle State** explicitly for reports,reflecting that reports evolve over time.
- Infrastructure discussions about Supabase Storage motivated separating **Image Asset** from **Report**.
- Differenciating lost items reports from found items reports, highligted the importance of

Report Type and semantic distinctions

- Roles mentioned in narratives(e.g. holder, person who lost an item, person who found an item), clarified **People Roles** and their responsibilities

Through this analysis, the **domain understanding** is validated by confirming that all key entities and events, roles and their relationships can realistically be captured.

3.2. Validation and Verification

The purpose of this section is to document how the team validates and verifies the domain concepts, requirements, and design decisions of the **Lost & Found** system.

3.2.1. Validation Strategy

Validation activities focus on confirming that the modeled concepts (**Report**, **User**, **Ownership**, **Lifecycle State**, **Image Asset**, etc.) correspond to real-world behavior and expectations. Rather than relying on abstract flows, validation is grounded in concrete, persona-driven scenarios that simulate realistic interactions.

These scenario walkthroughs are used during feedback sessions with students, teaching assistants, and administrative staff to surface overlooked edge cases and feature ideas. Each scenario step is explicitly linked to both functional and interface requirements, with expected outcomes and validation checks clearly defined.

Scenario 1: María — Reporting a Lost Laptop

María, a third-year Computer Science student, studies in the university library until the evening. The next morning, she realizes that her silver HP laptop is missing. She is unsure of the exact location where she left it, but recalls that it was likely in one of the study rooms she used.

She logs into the Lost & Found platform using her university credentials and selects "Report Lost Item." She fills in the form:

- Title: "Silver HP Laptop"
- Category: Electronics
- Approximate Time of Loss: "Evening, around 6:00 PM – 8:30 PM"
- Last Seen Location: "Library, 2nd floor, Study Room 3"
- Description: "Silver HP laptop with a small RUMblebots sticker on the back"
- Image Upload: Optional (María does not upload an image)

System behavior:

1. Validates required fields.
2. Binds María's `user_id` to the report.
3. Stores the report with status `"active"`.

Later, María remembers more details and edits the description:

- Adds: "Laptop has a faint bent near the upper right corner and a blue `intel core i7` sticker on the left side panel."

Non-owner edit attempt:

- Another student, Luis (her boyfriend), sees the report in the public feed and tries to edit the description to add a note to clarify that her laptop model is a HP Envy x360.
- The system denies the edit, logs the attempt, and optionally notifies Luis that only the report owner can make changes.

Validation Points:

Step	Requirements	Expected Outcome	Validation Check
Submit report	• [REQ-REPORT-F-01] • [REQ-REPORT-F-03] • [IR001] • [IR003]	Report is created, bound to María	Verify report exists with correct user ID and status
Edit own report	• [REQ-REPORT-F-03] • [IR005]	María successfully updates description	Check that edit is saved, timestamp preserved
Edit by other user	• [REQ-REPORT-F-03] • [REQ-CLAIM-F-01] • [IR009]	Luis denied edit	Attempt edit as Luis, verify system prevents changes

Insights and Actions:

- Confirmed enforcement of ownership rules.
- Preserved traceability of edits.
- Clarified workflow for non-owner edit attempts.

***Scenario 2: Javier — Reporting a Found Wallet ***

Javier, a student walking past the cafeteria around 1:30 PM, finds a brown leather walletF on the ground. He picks it up but notices it contains a student ID and other personal documents. He wants to report it without exposing sensitive information.

He logs into the Lost & Found platform and selects "Report Found Item." He fills in the form:

- Title: "Brown leather wallet"
- Category: Personal Items
- Found Location: "Cafeteria entrance, near the main doors"
- Found Time: 1:30 PM
- Description: "Wallet contains a student ID, some cash, and a few receipts. Sensitive information

not shown."

- Image Upload: A photo of the wallet exterior (without revealing contents).

System behavior:

1. Validates all required fields are filled.
2. Binds Javier's `user_id` to the found item report.
3. Stores the report with status `"active"`.
4. Automatically flags any sensitive information (e.g., ID numbers, full names) to prevent accidental exposure.

Later, Javier may update the description if he notices additional non-sensitive details, such as a subtle unique engraving, but no personal data is exposed.

Validation Points:

Step	Requirements	Expected Outcome	Validation Check
Submit found report	• [REQ-REPORT-F-02] • [REQ-REPORT-F-04] • [IR002]	Report is created in "active" state	Verify report exists in database, visible to search
Privacy handling	• [REQ-CLAIM-F-01] • [REQ-REPORT-F-02] • [IR004]	Sensitive information not exposed	Confirm no ID numbers or sensitive data stored in database or displayed
Search by keyword	• [REQ-SEARCH-F-01] • [REQ-SEARCH-F-02] • [IR014] • [IR015]	Report discoverable by authorized users	Perform search for keyword, verify report appears

Insights and Actions:

- Removed assumption of ID image uploads.
- Added privacy-preserving requirements.
- Reinforced minimal exposure of sensitive data.

Scenario 3: Automated Moderation and Review Workflow

A user submits a found or lost item report. The system automatically analyzes the text for suspicious language, inappropriate content, or potential policy violations (e.g., offensive words, spam, or attempts to share personal data publicly).

System behavior:

1. Automated content filter scans the report upon submission.

- If suspicious content is detected, the report status is set to `"flagged"`.
    - The report is hidden from public listings (quarantine) to prevent accidental exposure.
2. A human reviewer (moderator) receives the flagged report for evaluation.
3. Reviewer actions:
    - **Approve** → Report status changes to `"active"` and becomes visible to users.
    - **Remove** → Report status changes to `"removed"` and is not publicly accessible.
4. Author notification:
    - The system sends a message explaining the reason for flagging.
    - Author is informed of the outcome (approved or removed).
5. Appeal or edit:
    - If removed, the author may edit the report to remove flagged content and resubmit.
    - Appeals are tracked in the system for auditing and review.

This workflow ensures safety, privacy, and content compliance while allowing users to correct unintentional issues.

Validation Points:

Step	Requirements	Expected Outcome	Validation Check
Flag detection	• [REQ-ADMIN-F-01] • [IR018]	Report automatically flagged	Check system logs for flagged status
Visibility control	• [REQ-SEARCH-F-02] • [IR015]	Report hidden from public search	Perform search, verify report not visible
Author notification	• [REQ-ADMIN-F-02] • [IR017]	Author receives notification	Verify email or UI alert sent
Appeal/edit flow	• [REQ-ADMIN-F-02] • [REQ-REPORT-F-01] • [IR005]	Author can edit/appeal	Attempt edit/appeal, verify system allows corrections

Insights and Actions:

- Introduced automated moderation workflow.
- Added notification and appeal mechanism.
- Defined quarantine state before removal.

Scenario 4: Daniel — Searching for a Missing ID

Daniel realizes he has lost his student ID card. He logs into the Lost & Found platform to locate it.

Search steps:

1. Filters by category: **Personal Items**.
2. Enters keywords: `"ID"`.
3. Applies status filter: only `"Active"` reports.
4. System performs search based on textual descriptions and optional metadata, not relying on images.
5. Search results display:
    - Report title
    - Description
    - Location (if known)
    - Time window
    - Finder information (limited for privacy)
6. Daniel identifies a report that matches his ID and contacts the finder via the system's messaging interface.

This scenario demonstrates **text-based search capability**, **category filtering**, **status filtering**, and **privacy-conscious display of results**.

Validation Points:

Step	Requirements	Expected Outcome	Validation Check
Category filter	• [REQ-SEARCH-F-01] • [IR015]	Results filtered to selected category	Verify results show only matching category reports
Keyword search	• [REQ-SEARCH-F-02] • [IR014] • [IR015]	Reports matching keywords returned	Perform search, check matching results
No image dependency	• [REQ-REPORT-F-02] • [IR002]	Search works without images	Verify search still returns reports lacking images
Sorting / filtering by status	• [REQ-SEARCH-F-02] • [IR015]	Only "Active" reports shown	Confirm only active reports appear in search

Insights and Actions:

- Removed reliance on images.
- Reinforced text-based matching.
- Made images optional.

3.2.2. Verification Strategy

Verification activities focus on internal consistency, completeness, and traceability.

Peer Reviews

Each documentation subsection (Terminology, Concept Analysis, Requirements, Implementation Fragments) is reviewed by at least one team member not primarily responsible for writing it.

Verification checks include:

- Every requirement uses defined domain terminology.
- No undefined or ambiguous terms appear.
- Implementation fragments reflect documented architectural decisions (React + Supabase, RLS, storage separation).

During early review, an inconsistency was found between “delete” and “archive,” which was resolved by formally distinguishing lifecycle states.

Documentation Checklists

A structured checklist is applied at milestone review:

- Every domain concept appears in at least one requirement.
- Every requirement traces back to at least one project goal.
- Each requirement is testable and measurable.
- Ownership and RLS enforcement are documented both conceptually and technically.
- Lifecycle transitions are explicitly defined.

Traceability Matrix

A lightweight traceability matrix links:

- Goals → Requirements
- Requirements → Domain Concepts
- Requirements → Design Elements (database schema, RLS policies, React components)
- Requirements → Verification Method (test case, walkthrough, inspection)

Example:

Requirement: “Users can only edit their own reports.”

- Concept: Ownership
- Design: `user_id` bound to session + RLS policy
- Verification: Attempt cross-user modification test

Scenario-Based Verification Tests

Step	Requirement(s)	Expected Outcome	Verification Action
Submit valid report	• [REQ-REPORT-F-01] • [REQ-REPORT-F-03] • [IR001] • [IR003]	Report status = "active", bound to owner	Submit form as María; check database for correct `user_id` and status
Edit own report	• [REQ-REPORT-F-03] • [IR005]	Changes saved	Login as María, edit report, verify updated description and preserved timestamp
Edit as other user (active report)	• [REQ-REPORT-F-03] • [REQ-CLAIM-F-01] • [IR009]	Luis denied edit	Login as Luis, attempt edit, verify system prevents changes
Submit found report	• [REQ-REPORT-F-02] • [REQ-REPORT-F-04] • [IR002]	Report created in "active" state	Submit form as Javier; check database for status and owner binding
Privacy handling	• [REQ-CLAIM-F-01] • [REQ-REPORT-F-02] • [IR004]	Sensitive information not exposed	Confirm no ID numbers or personal info stored/displayed
Search by keyword	• [REQ-SEARCH-F-01] • [REQ-SEARCH-F-02] • [IR014] • [IR015]	Report discoverable	Perform search, verify report appears
Flag detection	• [REQ-ADMIN-F-01] • [IR018]	Report automatically flagged	Submit report with suspicious content; check logs/status
Visibility control	• [REQ-SEARCH-F-02] • [IR015]	Report hidden from public search	Search as any user, verify report not visible
Author notification	• [REQ-ADMIN-F-02] • [IR017]	Author receives notification	Verify email/UI alert sent to author
Appeal/edit flow	• [REQ-ADMIN-F-02] • [REQ-REPORT-F-01] • [IR005]	Author can edit/appeal	Login as author, edit removed report, submit, verify status updated

Step	Requirement(s)	Expected Outcome	Verification Action
Category filter	• [REQ-SEARCH-F-01] • [IR015]	Results filtered to selected category	Perform category-filtered search, verify only matching reports shown
Keyword search	• [REQ-SEARCH-F-02] • [IR014] • [IR015]	Reports matching keywords returned	Perform keyword search, check results
No image dependency	• [REQ-REPORT-F-02] • [IR002]	Search works without images	Verify search still returns reports lacking images
Sorting / filtering by status	• [REQ-SEARCH-F-02] • [IR015]	Only "Active" reports shown	Verify only active reports appear

Walkthrough Inspections

Date: `2026-03-04`

Purpose: Verify that the Lost & Found platform’s requirements, workflows, and system behavior match real-world expectations and are correctly implemented. This includes input validation, report lifecycle management, moderation workflows, access control, and search functionality.

Participants:

- **Reviewer:** Jean P. I. Sánchez Félix
  - Oversaw execution of scenarios, ensured traceability to requirements, logged observations
- **Students / Testers:**
  - Tasnim Said Khader
    - Executed scenario of reporting a lost laptop, performed edits, and attempted non-owner edits
  - Keishlyany M. Sanabria Santana
    - Executed scenario of reporting a found wallet, verified privacy handling and search visibility
  - Leanelys A. Gonzalez Orama
    - Executed moderation and appeal workflow scenarios, checked notification delivery and audit logging

Method:

- Each participant performed **step-by-step scenario walkthroughs** based on UI mockups and sample reports
- Reviewer observed interactions and logged system responses, noting deviations from

requirements

- Scenarios executed included:
    - **Reporting Lost Items** – submission with all required fields, editing by owner, attempted edits by non-owner
    - **Reporting Found Items** – submission with sensitive information, privacy compliance, optional image uploads
    - **Moderation and Review** – automated flagging, human review, approval/removal, notifications, appeals
    - **Search and Discovery** – filtering by category, keywords, and status; verifying text-based matches without images
- Checks were made against:
    - Functional requirements (mandatory fields, lifecycle states, search behavior)
    - Access control rules (owner vs. non-owner edits, moderation permissions)
    - Lifecycle state transitions (active → flagged → removed/approved)
    - Consistency of terminology in UI, instructions, and reports
    - Privacy and data handling (masking sensitive information, avoiding exposure)

Detailed Findings:

1. **Time Input Validation**
    - Observation: Participants attempted to submit reports without specifying time ranges. The system allowed submission but lacked validation
    - Impact: Could result in ambiguous or incomplete reports
2. **Moderation Workflow**
    - Observation: Flagged reports were quarantined, but the workflow did not support clear appeal or edit by the author
    - Impact: Could frustrate users and reduce trust in the system
3. **User Feedback on Flagged/Removed Reports**
    - Observation: Authors of flagged reports were not notified, leading to confusion about report status
    - Impact: Users unaware of issues could repeatedly submit problematic content
4. **Search Dependence on Images**
    - Observation: Search results prioritized reports with images; text-only reports were harder to discover
    - Impact: Reduces effectiveness of the search for items lacking images

Consequences / Actions Taken:

- **Time Range Validation**
    - Added mandatory time range input and validation messages to prevent incomplete

submissions

- **Moderation Workflow Improvements**
    - Implemented automated content flagging, quarantine, reviewer approval/removal, and appeal mechanism
    - All flagged reports now require human review before being publicly visible
- **User Notification System**
    - Added notifications for flagged or removed reports
    - Authors can edit and resubmit, and appeals are tracked in the system
- **Search Enhancements**
    - Enabled robust text-based search independent of images
    - Category, keyword, and status filtering reinforced

Findings and Actions

Finding	Action Taken
Missing or unvalidated time input	Added mandatory time range field with validation, prevents submission of incomplete reports
Unrealistic moderation workflow lacking appeal	Implemented automated moderation with quarantine, approval/removal, and clear appeal mechanism
No feedback to users about flagged/removed reports	Added notification messages, UI alerts, and edit/appeal flow for authors
Over-reliance on images in search	Enabled text-based search; results no longer dependent on images; filters applied for category and status

3.3. Requirements Elicitation and Stakeholder Analysis

3.3.1. Initial Stakeholder Identification

Domain Stakeholders and Inventory Lifecycle

Using the lecture slides as a starting point, this section identifies stakeholder groups, produces persona artifacts, maps responsibilities (RACI) across lifecycle states, and extracts stakeholder-driven user stories. The goal is to ensure that technical design decisions (data model, custody logs, claim verification, retention/disposal) reflect domain needs, safety concerns, and operational realities.

Scope and Objective

Scope: Align stakeholder perspectives with the Inventory and Item Lifecycle (reported → cataloged → stored → pending_claim → claimed → disposed).

Objective: Produce a stakeholder catalog, six persona cards, a RACI matrix, and at least three stakeholder-driven backlog items with acceptance criteria so these results can feed directly into sprint planning and risk mitigation work already completed.

Stakeholder Catalog

Stakeholder	Role / Interest
Finder (reporter)	Person who finds and reports an item. Wants quick reporting and secure handover; concerned about privacy and recognition.
Owner (lost person)	Person who lost the item. Wants rapid discovery, strong proof-of-possession verification, and privacy.
Operations / Intake Staff	Campus/venue staff who catalog items and manage storage. Needs clear custody logging and simple intake UI.
Campus Security / Police	Responsible for safety-sensitive items and managing secure handoffs. Needs audit trails and coordination with operations.
IT / DevOps	Maintains the app, storage, and services. Focuses on secure defaults, monitoring, and backups.
Legal / Records	Advises on retention/disposition rules, evidence handling, and compliance (e.g., institutional rules).
Volunteers	Assist with intake; may have limited privileges. Need clear role boundaries and training.
Users / Public	General users browsing lost items, reporting, or claiming. Expect privacy protections and usability.
Auditor / Compliance Officer	May review custody logs, retention, and disposal records for audits or disputes.
Third-party services (storage, SMS)	Provide infrastructure. Must follow secure config and least privilege.

Persona Cards

Each persona captures goals, fears, permissions, and 3 stakeholder-driven requirements (user stories).

Persona: "Fernanda — Finder (Student)"

- **Role:** Student who finds items around campus.
- **Goals:** Report quickly, attach photo, get anonymized acknowledgement.
- **Fears:** Being contacted directly, exposing her contact info, long paperwork.
- **Permissions:** Can create reports; cannot change custody logs.
- **Top user stories**
  - As a Finder, I want to upload photos and a rough location so staff can catalog the item.
    - **AC:** Image accepted, location attached, and confirmation sent to reporter with a reference ID.
  - As a Finder, I want my contact hidden from public listings.
    - **AC:** Finder contact is private and only used to communicate via ephemeral tokens.
  - As a Finder, I want to mark an item as "sensitive" if it contains IDs.
    - **AC:** Item flagged, staff notified for special handling.

Persona: "Carlos — Owner (Student)"

- **Role:** Student who lost an item.
- **Goals:** Find and claim my item quickly, with minimal proof required but secure.
- **Fears:** Fraudulent claim, privacy leaks.
- **Permissions:** Can initiate claim; cannot release item.
- **Top user stories**
  - As an Owner, I want to search by image or keywords so I can find matches.
    - **AC:** Search returns candidate items including image similarity if available.
  - As an Owner, I want to start a claim that sends a token to the reporter/staff.
    - **AC:** Token sent and claim marked pending for staff verification.
  - As an Owner, I want to see what proof is required for pickup.
    - **AC:** Claim page lists required proofs (e.g., serial number, photo of item).

Persona: "Ana — Intake Staff"

- **Role:** Staff member cataloging items at lost-and-found.
- **Goals:** Fast intake, accurate custody logs, minimal errors.
- **Fears:** Missing chain-of-custody entries, disputes.
- **Permissions:** Can set status, add custody entries, mark items stored/claimed.
- **Top user stories**
  - As Staff, I want a guided intake form ensuring required fields are filled.
    - **AC:** Form validation prevents incomplete intake submissions.
  - As Staff, I want to append custody entries with my staff ID.

        - **AC:** Each custody action logs actor_id and timestamp immutably.
    - As Staff, I want to flag items for legal review.
        - **AC:** Flagging workflow routes item to Legal queue.

Persona: "Mauricio — Campus Security Officer"

- **Role:** Security responsible for dangerous or high-value items.
- **Goals:** Ensure safe custody and coordinated handover.
- **Fears:** Unauthorized releases, contested claims.
- **Permissions:** Can place holds and require police-mediated pickup.
- **Top user stories**
    - As Security, I want to place an item on hold pending police clearance.
        - **AC:** Item status becomes 'hold' and prevents release until cleared.
    - As Security, I want to view full custody log for evidence.
        - **AC:** Full log accessible only to authorized roles with audit trail.
    - As Security, I want to request additional owner verification steps.
        - **AC:** System allows staff to require extra proofs and records their rationale.

Persona: "Diego — IT / DevOps"

- **Role:** Platform operator maintaining services and storage.
- **Goals:** Maintain uptime, secure storage, and logs.
- **Fears:** Misconfigured storage exposing images, privilege misuse.
- **Permissions:** Manage infra, rotate credentials.
- **Top user stories**
    - As DevOps, I want private object storage with presigned URLs to limit exposure.
        - **AC:** Images are stored private and presigned URLs expire within configured time.
    - As DevOps, I want alerts on unusual storage access patterns.
        - **AC:** S3 access logs forwarded and alerts fire on threshold.
    - As DevOps, I want role-based access control for staff and volunteers.
        - **AC:** RBAC configured and tested.

Persona: "Rosa — Legal / Records"

- **Role:** Legal advisor for retention and disposition.
- **Goals:** Ensure compliance with local laws and institutional policy.
- **Fears:** Improper disposals triggering liability.
- **Permissions:** Can require longer holds or special handling.
- **Top user stories**

- As Legal, I want configurable retention policies per category.
    - **AC:** Admin UI allows policy per category (A/B/C).
- As Legal, I want disposal approvals logged with approver id.
    - **AC:** Disposal is blocked without recorded approval and reason.
- As Legal, I want exportable custody logs for audits.
    - **AC:** Logs exportable as CSV with signatures.

RACI Matrix for Inventory Lifecycle Actions

Action	Finder	Owner	Staff	Security	IT/DevOps	Legal
Report item	R	A	C	C	I	I
Catalog item	I	I	R	C	I	I
Store / label item	I	I	R	C	I	I
Initiate claim	I	R	I	I	I	I
Verify and release	I	I	R (staff)	A (security if sensitive)	I	C
Dispose / donate	I	I	R (staff)	C	I	A (legal approval)

RACI legend: R = Responsible, A = Accountable, C = Consulted, I = Informed.

Stakeholder-Driven User Stories

1. **Mediated claim flow** (High priority) — *As an Owner, I want a secure token-based claim process that requires staff verification before release.*
    - **AC:** Token generated and validated; staff records two verification cues; custody log updated.
2. **Minimal PII exposure** (High) — *As a Finder, I want my contact hidden and communications mediated.*
    - **AC:** Finder contact not shown publicly; platform mediates messages via ephemeral tokens.
3. **Sensitive-item workflow** (Medium) — *As Security, I want the ability to mark items as 'sensitive' requiring police or staff-mediated pickup.*
    - **AC:** Sensitive items set hold and cannot be released without authorized clearance.
4. **Retention policy UI** (Medium) — *As Legal, I want configurable retention rules by category.*
    - **AC:** Admin UI to set days per category; expected_hold_until computed.

3.3.2. Stakeholder Discovery and Expansion

Stakeholder Persona Development and Elicitation Strategy

In order to connect domain engineering and requirements engineering, representative stakeholder perspectives were identified and modeled as personas. These personas capture goals, behaviors, and constraints within the UPRM lost and found domain. They serve as a foundation for deriving user stories and refining functional requirements.

Stakeholder Groups

Stakeholder	Role	Responsibilities	Primary Goals
Claimants	Individuals who lose items and attempt recovery.	Reporting lost items, searching for matches, verifying ownership.	Recover belongings quickly, minimize effort, protect personal information.
Finders	Individuals who discover misplaced items.	Reporting found items and providing descriptive details.	Return items to rightful owners with minimal friction.
Administrative Custodians	Authorized personnel responsible for oversight and moderation.	Reviewing reports, resolving misuse, maintaining data integrity.	Ensure trustworthy system operation and prevent abuse.

Personas

1. Student Claimant
   - **Name:** Carlos Rivera
   - **Age:** 20
   - **Role:** Undergraduate Student
   - **Background:** Carlos is a second-year engineering student who spends most of his time in classrooms, at the library or in study lounges. He frequently carries multiple items including a laptop, notebook, student ID and access cards.
   - **Goals:**
     - Quickly report lost belongings
     - Rearch for potential matches
     - Maintain privacy of personal information
     - Receive timely updates on recovery progress
   - **Day-in-the-life narrative:** Carlos realizes he left his calculator in a classroom after a lecture and he wants a simple way to report the loss and check whether someone has found it. He prefers not to publicly share personal contact details but wants to be notified if a matching item appears.
2. Administrative Custodian
   - **Name:** Maria Gonzalez

- **Age:** 38
- **Role:** Administrative Staff Member
- **Background:** Maria works in a campus office that frequently receives lost items. She manually records information and struggles to keep track of inquiries.
- **Goals:**
  - Monitor reports for accuracy
  - Prevent misuse or false claims
  - Ensure items are properly matched
  - Maintain organized tracking
- **Day-in-the-life narrative:** Maria receives a lost ID card and wants a reliable way to record it so the owner can find it. She also needs oversight tools to verify claims and update report status once the item is returned.

Elicitation Questionnaire

The following questions are designed to gather domain-specific information:

- **Participant Information**
  - Who is being interviewed?
  - What is their role in the lost and found process?
  - How frequently do they encounter lost items?
- **Item Discovery**
  - Where are items most commonly found?
  - What information is typically available when an item is discovered?
  - What actions are taken after finding an item?
- **Reporting Process**
  - How are lost items currently reported?
  - What information is necessary to identify an item?
  - What challenges occur during reporting?
- **Verification and Recovery**
  - How is ownership currently verified?
  - What information is used to confirm claims?
  - What happens after an item is returned?

Initial User Stories

US1: As a Student Claimant, I want to report a lost item so that others can help me recover.

US2: As a Finder, i want to report a discovered item so that it can be returned to its owner.

US3: As an Admininstrative Custodian, I want to rview reports so that I can prevent misuse.

US4: As a Student Claimant, I want my personal information kept private so that my data is protected.

3.3.3. Requirements Acquisition Planning

Domain Acquisition Strategy for the Lost & Found System

The domain acquisition strategy for the Lost & Found system focuses on structured elicitation of domain knowledge through stakeholder personas, observation extraction, and concept formation. This approach ensures that the resulting requirements are grounded in real-world behaviors and needs rather than abstract assumptions.

Domain Acquisition Approach

Rather than beginning with predefined entities or database structures, the acquisition process began with constructed stakeholder personas representing realistic university scenarios. These personas included lost item owners, finders, administrative staff, and environmental observers.

The process followed a staged refinement strategy:

1. Constructed Persona Narratives

Informal first-person accounts were created to simulate real-world campus experiences. These narratives captured behaviors such as delayed realization of loss, informal communication attempts, storage inconsistencies, and verification hesitation.

2. Observation Extraction

Each narrative was decomposed into explicit observations. Observations focused on factual behavioral patterns rather than interpretation. This prevented premature abstraction and preserved raw domain characteristics.

3. Derived Concept Formation

From repeated observations, reusable domain concepts were extracted (e.g., search ambiguity, retention window, platform fragmentation). This allowed patterns to emerge organically rather than imposing system assumptions too early.

4. Concept Consolidation

Recurring themes were analyzed to detect lifecycle inconsistencies, communication gaps, behavioral asymmetries, and trust-related concerns.

5. Formal Structuring

Only after concept stabilization were formal classifications introduced, including entity definitions, event definitions, behavioral constructs, and operational abstractions.

Contribution to Accuracy and Completeness

Accuracy was improved by capturing multiple perspectives and distinguishing observations from interpretations.

Completeness was improved by incorporating behavioral, spatial, communication, and lifecycle dimensions rather than focusing solely on object tracking.

The resulting domain description is therefore traceable to structured conceptual extraction rather than arbitrary system modeling.

Requirements Engineering Process Model for the Lost & Found System

The most appropriate Requirements Engineering (RE) process model for developing the university's Lost & Found platform is the Agile and Iterative Model. This methodological approach treats requirements not as a static document, but as a living product that evolves in parallel with the software development process.

Core Phases of the Agile Model

Instead of defining all functionalities at the start of the project, this model divides the process into development cycles and includes the following central phases:

- **Continuous elicitation**: Needs are constantly gathered through direct communication and feedback from users and administrators.
- **Incremental specification**: Requirements are documented using prioritized user stories in a product backlog, detailing only what is necessary for the next work cycle.
- **Iterative validation**: Prototypes or functional increments are built periodically for stakeholders to test, validate, and approve.
- **Dynamic requirement management**: The team maintains traceability and adapts requirement priorities based on the operational value each feature brings to the campus.

Justification Over the Sequential Model

A strict **sequential** approach, like the Waterfall model, assumes that requirements will not change, which is **unrealistic** in a dynamic university environment. The **iterative** model is superior for the Lost & Found app because it allows the development team to adapt to unforeseen factors without derailing the project. For example, if campus security updates its protocols for handling valuable or electronic items, the team can immediately integrate these new rules into the next cycle.

Furthermore, releasing early versions allows the team to observe how students actually **interact** with the reporting interface. This direct feedback ensures the final application is truly intuitive and solves the core problem, rather than blindly following an initial document that might be outdated.

Handling Changes During Development

In an iterative model, the evolution of requirements is an expected and planned advantage, not a methodological obstacle. If an administrator requests a new feature mid-development, such as adding an automated email notification system, the change process is structured yet highly flexible.

The development team captures the new request, analyzes its technical feasibility, and transforms it into a clear user story. This story is then inserted into the backlog, and its priority level is evaluated against the remaining work. If the administrator determines the feature is urgent, the team simply includes it in the planning phase for the very next programming cycle. In this way, the platform absorbs new institutional needs in a controlled manner, ensuring changes are implemented without disrupting the current architecture.

3.3.4. Requirements Acquisition Resources and Documentation

Requirements Acquisition Questionnaire

To acquire evidence for our domain claims, we designed and distributed a questionnaire to collect data about the lost-and-found experience on campus. This process involved both the creation of targeted questions and the collection of responses from relevant stakeholders. The goal was to gather concrete data that could inform our requirements decisions and support our claims about user needs and behaviors.

This section separates:

- **Acquisition** (the questions and data collection method)
- **Elicitation** (the interpretation of the collected responses)

Requirements Acquisition (Questionnaire)

Purpose

The questionnaire gathers evidence about:

- How often items are lost/found on campus (in the respondents' experience)
- What recovery steps people currently take (offices/places checked)
- What information people are willing to share publicly
- What risks people perceive (false claims, privacy concerns)
- Which improvements would make recovery easier

Method (Acquisition)

- Collection method: Google Form (recommended) or direct messages.
- Target stakeholders: students (lost-item owners + finders) and optionally staff who handle found items.
- Time window: Mar/28/2026 - Apr/5/2026
- Respondents: N = 10

Questionnaire

ID	Question
Q1	In the last **6 months**, how many times have you lost an item on campus? `(0 / 1 / 2–3 / 4+)`
Q2	What items did you lose? `(keys / ID / earbuds / bottle / charger / laptop / other: _)`
Q3	When you realize you lost something, what is the **first** thing you do? `(retrace steps / ask friends / check office / group chat / other)`
Q4	In your last loss, which places/offices did you check? `(library / security / department office / student affairs / cafeteria / other: _)`
Q5	About how many places did you check before you found it or gave up? `(1 / 2 / 3 / 4+)`
Q6	How long after losing something do you usually notice it's missing? `(immediately / hours later / next day / other)`
Q7	If you have found an item on campus, what did you do? `(left it / turned in to office / posted in chat / kept it / other)`
Q8	If you turned in a found item, where did you take it? `(library / security / department office / student affairs / other)`
Q9	What details help you identify your item **without** making it easy to fake-claim it? `(open response)`
Q10	What details would you **not** share publicly? `(name / phone / ID details / exact location / photos / other)`
Q11	Have you ever seen/heard of someone falsely claiming an item? `(Yes/No)`. If yes, describe briefly.
Q12	What is the biggest obstacle in the current lost-and-found process at UPRM? `(open response)`
Q13	If you could change one thing about how lost & found works on campus, what would it be? `(open response)`

Elicitation Summary

Method

- Respondents: N = 10
- Stakeholder mix: 8 students, 2 staff (security)
- Collection method: Google Form
- Dates: March 16, 2026 to April 3, 2026
    - Small sample size
    - Limited representation of staff perspectives

Findings

- F1 (Frequency): 8/10 respondents reported losing at least one item in the last 6 months.
- F2 (Most common items): Top items mentioned were: 1) water bottle 2) charger 3) earbuds (tied

with keys)

- F3 (Time-to-realization): Most respondents noticed loss: hours later.
- F4 (Recovery effort): Median number of places checked before finding/giving up: 2.
- F5 (Where people check): Most commonly checked places were: library, department offices, and group chats.
- F6 (Finder behavior): 10/10 respondents reported finding an item at least once; common actions were: turning it in to an office (most common), posting in a chat, or leaving it where it was.
- F7 (Privacy/trust): Common details people avoid posting publicly: phone/contact info, ID details, and exact locations (also serial numbers/photos of IDs); 5 respondents mentioned false-claim concerns.

What These Findings Support

Based on the findings above, we can support the following claims:

- C1: Lost items occur often enough (in our sample) to justify improving the recovery process.
- C2: Recovery currently requires checking multiple places/channels (place-to-place effort).
- C3: Privacy concerns exist and influence what information people are willing to share publicly.
- C4: Finder behavior is inconsistent, suggesting the need for clearer guidance and safer handoff/verification.
- C5: False claims are a concern for some users, indicating the need for robust verification and trust-building features in the system.

3.3.5. Requirements Traceability Framework

Traceability for Report Lost Item

Report Lost Item Requirement

Reporting Requirements:

- The system must accept only **Reports** that include sufficient descriptive attributes to support **Matching**, including at minimum:
  - item category
  - textual description
  - approximate location (loss or discovery)
  - relevant date or time window
- The ID for a Lost Item Report: [REQ-REPORT-F-01]
- **Scope:**
  - The requirement : [REQ-REPORT-F-01], must be traced across:
    - User Stories, Epics and Features
    - Domain Requirements

- Interface Requirements
- Machine Requirements
- Test and Verification artifacts

Traceability Policy Rules:

- The Lost Item Requirement: [REQ-REPORT-F-01], much be linked to all related artifacts
- Traceability links must be maintained and updated during evolution
- Any verification is traced back to [REQ-REPORT-F-01] to confirm it is satisfied
- Any modification or change to [REQ-REPORT-F-01] must trigger impact analisys
- The requirement is fully traceable when all relevant interfaces, requirements and tests are captured and linked in the traceability table.

Traceability Links Between the Requirement and Related System Components

Backward Traceability:

- [REQ-REPORT-F-01] - Submit a Lost Item Report
- Epic E1: Reporting Lost and Found Items
- Feature F1.1 - Structured lost item submission
- Feature F1.3 OptionalImage upload support
- Feature F1.4 Edit and manage own reports
- Feature F1.5 - Automatic ownership binding

The Report Lost Item requirement interacts with multiple components of the system:

Domain Requirements

- **Reporting Requirements**
  - [REQ-REPORT-F-01]: The system shall allow a registered UPRM community member to create a Report for a Lost Item, including sufficient descriptive attributes to support Matching (e.g., category, description, approximate location, date or time window).
  - [REQ-REPORT-F-02]: Each Report shall distinguish between public descriptive information and private verification information within each Report.
  - [REQ-REPORT-F-03]: Upon creation, each Report shall be assigned an initial status of "Active".
  - [REQ-REPORT-F-05]: The system shall associate each Report with its creator to support ownership, traceability, and controlled modification.
- **Search and Discovery Requirements**
  - [REQ-SEARCH-F-01]: The system shall allow users to search Reports based on keywords related to item description, category, approximate location, and date or time window.
  - [REQ-SEARCH-F-02]: The system shall allow filtering of Reports by category, general location,

date, and status.
    - [REQ-SEARCH-F-03]: The system shall present search results in a manner that supports comparison between Lost Item and Found Item Reports for Matching purposes.
- **Matching and Tracking Requirements**
    - [REQ-MATCH-F-01]: The system shall support the identification of potential matches between Lost Item and Found Item Reports based on descriptive attributes.
    - [REQ-MATCH-F-04]: The system shall allow the Owner of a Lost Item to monitor the status of their Report (Active, Returned, Closed, Expired).
    - [REQ-MATCH-F-03]: The system shall record status transitions from Active to Returned, Closed, or Expired.

Claim Verification and Recovery Requirements

- [REQ-CLAIM-F-01]: The system shall restrict marking an item as Returned to authenticated users involved in the Report.
- [REQ-CLAIM-F-02]: The system shall support the use of private verification information to validate ownership before Recovery is confirmed.
- [REQ-CLAIM-F-03]: Once Recovery occurs, the corresponding Report shall transition to "Returned" status.

Closure Requirements

- [REQ-CLOSE-F-01]: The system shall allow a Report to be manually closed if Recovery is no longer expected.
- [REQ-CLOSE-F-04] / [REQ-CLOSE-F-05]: Closed Reports shall remain recorded for reference but shall be clearly distinguishable from Active Reports in search results.

Administrative Oversight Requirements

- [REQ-ADMIN-F-01]: The system shall allow authorized administrators to review Reports for inappropriate or misleading content.
- [REQ-ADMIN-F-02]: Administrators shall be able to modify or remove Reports that violate institutional guidelines.
- [REQ-ADMIN-F-03]: Administrators shall be able to override Report status in cases of misuse or verified dispute.

Interface Requirements:

- [IR001] : The system must provide a public view for item details showing only public fields (category, description, photos, general location) and a prompt to log in to contact the owner
- [IR003] : Report creation forms (Lost/Found) must include structured fields with validation, such as category dropdown, description, location picker, photo uploader, and private fields.
- [IR005] : Each report must allow the Owner or Finder to update descriptive attributes, with the system replacing previous values and maintaining a history of modifications.

Machine Requirements:

- [SWR-004] : The system must implement form validation using React Hook Form with Zod or Yup schema validation.
- [SWRQ-005] : The system must support image upload functionality with file size limit of 10MB per image for JPEG, PNG, and GIF formats.
- [ENVRQ-006] : The system must implement Row Level Security (RLS) policies in PostgreSQL for all data access.
- [ENVRQ-009] : The system must implement rate limiting on all API endpoints to prevent abuse

Dependent Systems:

- Search and Filtering Systems
- Matching System
- Notification System
- Claim and Recovery System

Change Scenarios

1. Adding a New required field such as exact location or timestamp

Effect:

- [IR003] form must be updated
- [IR001] items public view must be updated
- [SWR-004] validation rules must be updated
- Database(Supabase) schema must be updated
- Search and filtering functionality affected

2. Change in Image Upload policy such as reducing file size limit or allowing more image formats

Effect:

- [SWR-005] must be updated
- [IR001] UI must reflect new constraints
- API validation must be modified

3. Changing data Retention Policy such as auto-delete after a year

Effect:

- Search and Filtering systems must exclude expired reports
- [ENVRQ-006] must be updated
- New requiremnet for scheduled deleition must be introduced
- [IR001] must exclude old reports from view

- [IR003] UI must inform users about report expiration
- Database(Supabase) schema must be updated
- Matching system impacted due to loss of historical data

3.3.6. Stakeholder Documentation and Justification

Example User Scenarios for the Lost & Found System

This section describes example user interaction scenarios for the Lost & Found system. These scenarios illustrate common behaviors within the platform, including reporting lost items, posting found items, submitting claims, and reviewing claims.

Introduction

The Lost & Found system supports multiple types of user interactions related to reporting, searching, and recovering items. The following scenarios present common flows that help define how users and administrators are expected to interact with the system.

Scenario 1: Reporting a Lost Item

Actor: User

1. A user logs into the Lost & Found platform.
2. The user selects "Report Lost Item."
3. The user enters item details such as description, location, and date lost.
4. The system saves the lost item report and makes it searchable.

Result: The lost item is stored in the system and can later be matched with found items.

Scenario 2: Posting a Found Item

Actor: Finder (User)

1. A user finds an item on campus.
2. The user logs into the platform.
3. The user selects "Post Found Item."
4. The user enters details about the item and where it was found.
5. The system publishes the found item listing.

Result: The found item becomes visible to users searching for lost items.

Scenario 3: Submitting an Item Claim

Actor: User

1. A user searches the platform for a lost item.

2. The user finds a listing that matches their item.
3. The user submits a claim request.
4. The system records the claim and sends it for review.

Result: The claim is stored in the system and awaits verification.

Scenario 4: Admin Reviews a Claim

Actor: Administrator

1. The administrator reviews submitted claim requests.
2. The administrator verifies the claim information.
3. The administrator decides whether the claim is valid.
4. The claim is approved or rejected.

Result: If approved, the item can be returned to the owner.

3.3.7. Requirements Acquisition Artifacts

Formulate Domain Narratives for Key UI Workflows

In accordance with the domain analysis process, raw domain description units must be abstracted into formal concepts and subsequently rewritten into a cohesive domain narrative. Before translating the UI workflows, we must establish the formalized domain terminology abstracted from the UPRM Lost & Found environment:

- **Property**: The abstract concept representing physical objects (e.g., phones, keys, water bottles) that are either lost or found.
- **Actors**: The individuals interacting with the system, further classified into Finders (those who locate Property) and Owners (those who lost Property).
- **Custodial Hubs**: The abstracted concept for designated physical locations where Property can be securely surrendered or claimed (e.g., Guardia Universitaria, Library Front Desk, Department Offices).
- **Match Event**: A dynamic reactive occurrence where the system correlates a Lost Property report with a Found Property report.

Domain Narratives for Key UI Workflows:

Workflow A: Reporting and Surrendering Found Property

- **UI Interaction**: The user clicks the "Report Found Item" button, fills out a form specifying the item’s category, location found, and photo. The user then selects a radio button indicating they will drop it off at Campus Security, and clicks "Submit".
- **Domain Narrative**: The Finder interacts with the system to record a newly discovered unit of Property. The Finder provides the intrinsic attributes of the Property (category, visual evidence, and discovery location) into the system. Next, the Finder selects a specific Custodial Hub where

they intend to physically surrender the Property. Upon submission, the system registers the Found Property and transitions its state to pending surrender at the designated Custodial Hub.

Workflow B: Smart Match Notification and Claim Initiation

- **UI Interaction**: A student receives a push notification saying "Potential Match Found!". They tap the notification, opening a detailed view of a found item currently located at the Library. They click the "Initiate Claim" button to alert the system they are on their way to pick it up.
- **Domain Narrative**: The system autonomously evaluates records and generates a Match Event between a recorded Lost Property and a newly registered Found Property. The system acts dynamically and reactively to alert the Owner of the Match Event. The Owner accesses the interface to review the Match Event details, which display the attributes of the Found Property and the specific Custodial Hub currently holding it. Concluding the flow, the Owner submits a formal claim request, signaling their intent to travel to the Custodial Hub to verify ownership and retrieve the Property.

3.4. Requirements Analysis and Conflict Resolution

3.4.1. Inconsistency Analysis

Requirements Inconsistencies

A) Types of Requirements classes identified on the project

After reviewing the project documentation, the following requirements clashes were identified:

Terminology clash

NOTE	Terminology clash: same concept named differently in different statements

In the project Requirements there are some terminology clashes:

1. The use of "registered UPRM community member" vs. "users" to refer to the same concept.

Important Note: All users of the system must be authenticated via the institutional email upr.edu, therefore: All users are any person authenticated via the institutional e-mail: upr.edu who are part of the Mayagüez Campus

Examples in the Requirements:

Reporting Requirements:

> "The system shall allow a registered UPRM community member to create a Report for a Lost Item."

vs.

Interface Requirements:

> "The login page must initiate the UPR.edu authentication redirect and inform users about required access for interactions."

Candidate Solution:

- Clarify the definition of "user" in the terminology section of the project documentation User : Any person authenticated via UPR institutional e-mail : upr.edu who is part of the Mayagüez Campus(Universidad de Puerto Rico recinto de Mayagüez)
- Replace all occurences UPRM community member in the requirements with "users" Example:

Before: Reporting Requirements:

"The system shall allow a registered UPRM community member to create a Report for a Lost Item."

After: The system shall allow a registered **user** to create a Report for a Lost Item

Designation Clash

NOTE	Designation clash: same name for different concepts in different statements

After reviewing the requirements and the terminology section of the project documentation, no clear designation clashes were identified. All terms are used consistently according to their definitions.

Structure clash:

NOTE	Structure clash: same concept structured differently in different statements

1. General Location is used in a very ambiguous and inconsistent way, across different parts of the requirements. The general location could refer to different locations. In terms of the system it could also refer to different location representations : free text input, map location or predefined location points.

For example:

Interface Requirements:

> "The system must provide a public view for item details showing only public fields (category, description, photos, general location) and a prompt to log in to contact the owner."

and

> "Report creation forms (Lost/Found) must include structured fields with validation, such as category dropdown, description,location picker, photo uploader, and private fields."

Candidate Solution:

- Define the term "General Location" in the Terminology section with a consistent meaning Keeping in mind that general location could refer to the place an item was lost/found or the place the found item can be retrieved To do so, define sub categories of general location.

Example:

- Lost/Found Location: location where the item was lost or found
- Pickup/ return location: location where the owner or finder can physically retrieve or pickup an item (e.g. Library, student center, cafeteria, common areas)
- Replace ambigous mentions, and apply the new different terms on the requirements

Strong Conflict:

NOTE	Strong conflict: statements not satisfiable together

After reviewing the project requirements, no strong conflicts were identified. In general, no requirement contradicts or prevents the implementation of other requirement.

Weak Conflict

NOTE	Weak conflict (divergence): statements not satisfiable together under some boundary condition

Important Note: We are going to go more in depth for this type of requirement clash on the following point.

Weak conflicts can be seen on the administrative oversight requirements.

> "Administrators shall be able to override Report status in cases of misuse or verified dispute."

vs.

> "Closed Reports shall remain recorded for reference but shall be clearly distinguishable from Active Reports in search results."

B) Weak Inconsistencies Requirements and boundary conditions

Weak Inconsistencies	Boundary Condition	Possible Risk	Possible Solution
1. "Administrators shall be able to override Report status in cases of misuse or verified dispute." vs. "Closed Reports shall remain recorded for reference but shall be clearly distinguishable from Active Reports in search results."	Administrator opens a closed report	Report status may cause confusion. Traceability changes may also occur	Restrict overrides and audit them.Clearly notify/show the users the overridden status in the UI
2. "The system shall record status transitions from Active to Returned or Closed." vs. "Administrators shall be able to override Report status in cases of misuse or verified dispute."	Administrator changing report status without a verified reason	Inconsistency between the system report status and the actual report lifecycle	Justification must be provided for manual status changes. The user must be notified of every change and the reason. Use timestamps for traceability
3. " The system shall record status transitions from Active to Returned or Closed." vs. "The system shall allow a Report to be manually closed if Recovery is no longer expected"	If a report is manually closed, the system will have a Report that transitions from Active to Closed, without passing through returned	Inconsistent status lifecycle	Update workflow rules to allow(In such specific cases) direct Active to Closed transitions. Clearly identify this exceptions

Weak Inconsistencies	Boundary Condition	Possible Risk	Possible Solution
4. "The system shall distinguish between public descriptive information and private verification information within each Report." vs. "The system must provide a public view for item details showing only public fields (category, description, photos, general location) and a prompt to log in to contact the owner." vs. "The system shall allow users to search Reports based on keywords related to the item description"	The system fails to separate public from private information or the user accidentally includes private verification information on the item description.	Private information leaked due to improper separation of public and private data	Clearly identify and label fields public and private. Inform users that important private information needs to be on the specified fields otherwise their private information could be leaked or could lead to scams.Validate user input and enforce strict field level access control

Weak Inconsistencies	**Boundary Condition**	**Possible Risk**	**Possible Solution**
5. "The system shall distinguish between public descriptive information and private verification information within each Report." vs. "The system shall present search results in a manner that supports comparison between Lost Item and Found Item Reports for Matching purposes." vs. "The system shall support the identification of potential matches between Lost Item and Found Item Reports." vs. "The system shall allow a Report to be manually closed if Recovery is no longer expected." vs. "Closed Reports shall remain recorded for reference but shall be clearly distinguishable from Active Reports in search results."	Closed reports information is included in the matching algorithm and search results alongside Active reports	Users or the system may attempt to match a closed report	Exclude closed reports from keywords, search and matching suggestions. Visually distinguish (it could be by using a label or just by hiding the report from user view) closed from open reports.

C) Further Elicitation

To better understand and resolve the identified requirements inconsistencies the following elicitation techniques are required:

- Interview with stakeholders to help us answer and determine the following questions: What are their needs? , What do they want/expect from our system/platform?
- Meetings and Brainstorming sections between the project team to clarify important concepts and keep consistency around requirements(and documentation in general)

Further elicitation for identified(3) weak inconsistencies cases :

NOTE	The following questions represent an initial draft used to guide analysis. They were improved and simplified for stakeholders clarity, removing some technical aspects/terms. The simplified version is presented on section E) "Questionarie to Stakeholders"

Case 1:

Conflict:

> "Administrators shall be able to override Report status in cases of misuse or verified dispute."

vs.

> "Closed Reports shall remain recorded for reference but shall be clearly distinguishable from Active Reports in search results."

Questions to ask to the stakeholders in the interview:

- Under what circumstances should an administrator override the report status?
- How can the original report user be notified of the override? Will the reasons of the override be clearly detailed and disclosed ?
- Should the system keep a history of changes(logs) of all status changes, clearly identifying by who the changes were made?
- How should overwritten reports appear on the system(e.g. search results)? Should they have a special label or flag? or should they just appear as any other report? Do other users navigating the platform know that report was overwritten? Will those external users, know the causes of the overwritten? Will the reasons be shown or hide to the users?
- If a "Closed" report is Re-open, how is this going to impact/affect the matching algorithm and search results?
- After an override will there be a new status such as "Overridden" or will they remain as (e.g. "Active" , "Closed")

Case 2:

Conflict:

> "The system shall record status transitions from Active to Returned or Closed."

vs.

> "The system shall allow a Report to be manually closed if Recovery is no longer expected"

For this case conflict, I am also referencing this other requirement: "Administrators shall be able to override Report status in cases of misuse or verified dispute."

Questions for stakeholders:

- Should there be conditions before a user is able to manually close a report?
- If a user wants to close a report without it being returned, in other words, if the status of the reports goes from "Active" to "Closed" without being "Returned", how is the system expected to manage such cases?
- Under what conditions could that change of states be possible(if the previous question was affirmative)? Do users need a special permission or will administrators override the report status?
- Should users provide a reason for why they want to close an issue without it being returned?
- In such cases, who is allowed to manually close the report administrators or users?

Case 3:

Conflict:

> "The system shall distinguish between public descriptive information and private verification information within each Report."

vs.

> "The system must provide a public view for item details showing only public fields (category, description, photos, general location) and a prompt to log in to contact the owner."

vs.

> "The system shall allow users to search Reports based on keywords related to the item description"

Questions for stakeholders:

- What fields are considered as private verification information?
- How should users be warned if they accidentally expose private information?
- In such cases, how will the system find/validate if the information is private or public?
- How would the system determine if private information is exposed?
- What will be the corrective actions once the system finds that private information has been made public? How is the system going to protect this private information if exposed? What is the corrective pan?

D) Resolutions Tactics and Risk Reduction Techniques

Based on Lecture Notes, the following tactics could be used to resolve or handle weak conflicts:

1) Administrators override report status

Conflict resolution tactic: Weaken conflicting statements

Solution:

- Allow overrides only under verified disputes, with mandatory justification and user notification disclosing the reasons of the override
- Keep a history of changes(log) to audit all the changes with timestamps

Risk reduction tactic: Reduce Risk likelihood Reason: The recommended solution decreases the risk of confusion by providing justifications with timestamps, visible to the report creator.

2) Active to Returned to Closed vs. manual closure

Conflict resolution tactic: Restore conflicting statement

Solution:

- Allow direct Active to Closed transitions under specified conditions and flag these exceptions in the system.

Example of what could this expecified conditions exceptions be :

- A user finds the reported lost item without it being reported as found in the website/system. In other words, without the help of the system. This is a valid reason for skipping the "Returned" state

Risk Reduction Tactic: Avoid Risk Reason: The recommended solution avoids the risk by clearly defining the conditions under which manual closure is allowed, flagging and allowing these exceptions in the system.

3) Public information vs. Private fields

Conflict resolution tactic: Introduce new requirement

Solution:

- Use strict-field level access control, labeling and validation. Users cannot enter private information on public fields. Notify the users if necessary to avoid them accidentaly posting private information un public fields like item description, before submission.

Risk reduction tactic: Reduce consequence likelihood Reason: This proposed solution, validates the information and notify the users before submission.

4) Closed reports appearing in matching/search

Conflict resolution tactic: Avoid boundary condition

Solution:

- Exclude those reports from matching algorithms and match suggestions, while keeping them visible in search results for reference, in compliance with the requirement: "Closed reports shall remain recorded for reference"
- Visually separate them in search results
- Flag them or use a label, to let users know that the report is closed

Risk Reduction Tactic: Avoid risk consequence Reason: Closed reports are excluded from matching and keywords. Only visible in search results for reference

5)Terminology : User vs UPRM community members

Conflict resolution tactic: Specialize conflict source

Solution:

- Define User = authenticated UPRM community member
- Replace all inconsistent terms

Risk Reduction tactic: Mitigate risk consequence Reason: all references to users are standardized as "authenticated UPRM community members" in documentation to prevent misinterpretation.

E) Questionarie to Stakeholders

Questionarie questions are based on the 3 cases of weak inconsistencies identified on point C : "Further elicitation for identified(3) weak inconsistencies cases ": The questions were improved for clarity, removing technical terms and splitting them to facilitate the recovery of information and avoid stakeholders confusion.

Stakeholder Questionnaire: Lost and Found project

General Questions:

1. What are your main expectations for the Lost and Found website?
2. Are there any essential features you consider critical?

(The following questions were obtained from identified weak inconsistencies in requirements)

A) Report status override

Context:

- Administrators can override status
- Closed reports must remain visible but distinguishable

Questions:

1. Under what circumstances should an administrator be allowed to override a report status?
2. Should users be notified when the report status is overriden?
3. If yes, how can the user be notified of the override? Will the reasons of the override be clearly detailed and disclosed ?
4. Should the system keep a history of changes(logs) of all status changes, clearly identifying by who the changes were made?
5. How should overwritten reports appear on the system(e.g. search results)? Should they have a special label or flag? or should they just appear as any other report?
6. Should other users be able to see the reason for the override?
7. After an override, should there be a new status such as "Overridden" or should the reports reuse existing statuses(e.g. "Active" , "Closed")

B) Manual closing Lost or Found Reports

Context:

- Reports lifecycle goes from Active(Open report) to Returned to Closed
- Users may manually close reports

Questions:

1. Should there be any conditions before a user is able to manually close a report?
2. If a report is closed without it being returned, how should the system handle it?
3. Do users need a special permission or should administrators override the report status?
4. Should users be required to provide a reason when closing a report?
5. Who should be allowed to manually close the report, administrators , users or both?

C) Public vs Private Information

Context:

- Reports contain public and private information
- Users can search items based on keywords
- Users can view item details from public fields such as category, description, photos

Questions:

1. What information should be considered private?
2. Should the system warn users if they accidentally expose private information?
3. In such cases, how should the system automatically detect if the information is private or public?
4. How should the system determine if private information is exposed?
5. What should be the corrective actions once the system finds that private information has been made public?
6. How should the system protect private information if it is exposed?

3.4.2. Competing Goals and Derived Requirements

This section identifies competing goals derived from previously analyzed weak inconsistencies. These goals represent valid but opposing domain needs that cannot be simultaneously satisfied without introducing constraints. Each conflict is resolved through explicitly defined derived requirements grounded in domain behavior, traceability, and system consistency.

Administrative Flexibility vs. System Integrity

Domain basis: Administrative actors require the ability to intervene in reports to resolve disputes, misuse, or incorrect states. At the same time, reports represent a structured lifecycle whose integrity must remain consistent, auditable, and reliable for all stakeholders.

Competing goals: Administrators need sufficient flexibility to correct errors, enforce policy, or resolve disputes, which sometimes requires changing the state of a report outside its normal lifecycle. At the same time, the system must maintain consistent, traceable records of report states so that users and auditors can trust the historical record. Allowing unchecked administrative overrides could compromise the lifecycle's integrity, but over-restricting administrators could prevent timely resolution of critical issues. The tension lies in balancing these two needs: enabling effective administrative intervention without undermining system reliability.

Decision: Administrative intervention is permitted but constrained through traceability, justification, and visibility requirements.

Requirements:

- REQ-DERIVED-ADMIN-01: The system must require explicit justification for any administrative override of a **Report** state.
- REQ-DERIVED-ADMIN-02: The system must record all administrative overrides, including:
    - timestamp
    - administrator identity
    - previous state
    - new state
    - justification

- REQ-DERIVED-ADMIN-03: The system must notify the associated **Report** owner when an administrative override occurs.
- REQ-DERIVED-ADMIN-04: The system must preserve all previous state transitions and prevent deletion or modification of historical records.

User Autonomy vs. Workflow Consistency

Domain basis: Users may abandon recovery efforts or recover items outside the system. However, reports follow a structured lifecycle representing real-world recovery processes that must remain interpretable and consistent.

Competing goals: Users may need the ability to close a report early or skip stages of the recovery process if the item is recovered externally, abandoned, or otherwise resolved. However, skipping steps in the standard lifecycle reduces consistency and could confuse other users, administrators, and automated workflows. The system must reconcile the need for user autonomy with the need to preserve a coherent, auditable report history that accurately reflects the real-world sequence of events.

Decision: Controlled deviations from the lifecycle are allowed under explicit conditions and must remain traceable.

Requirements:

- REQ-DERIVED-LC-01: The system must allow direct transition from **Active** to **Closed** only under explicitly defined conditions.
- REQ-DERIVED-LC-02: The system must require a justification when a **Report** is manually closed without passing through the **Returned** state.
- REQ-DERIVED-LC-03: The system must flag all **Reports** that deviate from the standard lifecycle sequence.
- REQ-DERIVED-LC-04: The system must preserve the complete history of lifecycle transitions for every **Report**.

Information Visibility vs. User Privacy

Domain basis: Effective search and matching require visibility of item attributes. Ownership verification requires restricting sensitive information to prevent false claims and misuse.

Competing goals: Maximizing visibility of item data improves the likelihood of successful matches and recovery. However, exposing private or sensitive attributes risks misuse, privacy violations, or fraudulent claims. The tension lies in providing enough public information to facilitate discovery while strictly controlling sensitive details for verification purposes. Mismanaging this balance could either reduce recovery success or compromise user privacy.

Decision: Strict separation, validation, and controlled exposure of information are enforced.

Requirements:

- REQ-DERIVED-PRIV-01: The system must enforce separation between:

    - public descriptive attributes
    - private verification attributes
- REQ-DERIVED-PRIV-02: The system must validate input to prevent private verification information from being included in public fields.
- REQ-DERIVED-PRIV-03: The system must notify users when potential private information is detected in public fields prior to submission.
- REQ-DERIVED-PRIV-04: The system must restrict access to private verification attributes to authorized actors only.

Historical Completeness vs. Matching Accuracy

Domain basis: Closed, returned, and expired reports provide valuable historical information. However, including them in active matching introduces irrelevant or incorrect associations.

Competing goals: Preserving historical completeness ensures that the system provides a reliable record of all reported events, supporting auditing, analytics, and trend analysis. Conversely, including outdated reports in active matching could generate false positives, misleading users, or clutter recovery workflows. The system must balance retaining complete history with maintaining high relevance and accuracy in operational processes.

Decision: Historical data remains accessible but is excluded from active recovery processes.

Requirements:

- REQ-DERIVED-MATCH-01: The system must exclude **Closed**, **Returned**, and **Expired** reports from matching algorithms and match suggestions.
- REQ-DERIVED-MATCH-02: The system must distinguish non-active reports from **Active** reports in all search and retrieval results.
- REQ-DERIVED-MATCH-03: The system must allow non-active reports to remain accessible for reference without participating in active recovery workflows.

Terminology Consistency vs. Expressiveness

Domain basis: Multiple terms may refer to the same concept (e.g., “user” vs. “UPRM community member”), increasing ambiguity and reducing clarity across requirements.

Competing goals: Stakeholders may use different language to describe the same concept, reflecting different perspectives or organizational culture. However, inconsistent terminology can introduce misinterpretation, confusion, and errors in system design and communication. The system must enforce a single standardized vocabulary while accommodating necessary expressiveness in user-facing contexts.

Decision: A single standardized terminology is enforced across all documentation.

Requirements:

- REQ-DERIVED-TERM-01: The system documentation must define a single standardized term for each domain concept.

- REQ-DERIVED-TERM-02: All requirements must consistently use the standardized terminology defined in the terminology section.
- REQ-DERIVED-TERM-03: The system must define **User** as an authenticated UPRM community member identified via institutional email.

3.4.3. Conflict Resolution Strategies

Data Consistency and Conflict Resolution in the Lost & Found System

In multi-user systems, maintaining data consistency is essential to ensure that information remains accurate and reliable despite concurrent interactions. In the Lost & Found platform, users may simultaneously submit reports, update item information, or initiate claim requests, which introduces the risk of conflicting or inconsistent data.

Assumptions (Policy / System)

- The platform allows users to submit Lost and Found item reports.
- Users may update item details unless restricted by system state.
- Claims follow a structured process (request → verification → approval/rejection).
- Admin or staff roles are available to resolve conflicts and enforce consistency.
- If system implementation differs, expected behaviors should be adjusted accordingly.

Conflict Scenarios and Expected Behavior

CS-01: Duplicate item reports

- **Scenario**: Multiple users submit reports describing the same lost or found item.
- **Risk**: Data duplication and confusion for users searching the platform.
- **Expected behavior**:
  - System flags potential duplicates based on similarity (description, location, date).
  - Staff/Admin reviews and merges or links duplicate entries.
  - One canonical listing is maintained while others are marked as duplicates.
  - **Further elicitation**: Staff/Admin requests additional details from reporters (e.g., distinguishing features, exact location, timestamps) to determine whether reports refer to the same item before merging.
- **Roles involved**: Reporters, Staff/Moderator, Admin.

CS-02: Multiple users claiming the same item

- **Scenario**: Two or more users submit claims for a single item.
- **Risk**: Conflicting ownership decisions and possible fraud.
- **Expected behavior**:
  - Item enters Pending Verification after the first claim.

    - Additional claims are allowed but marked as queued.
    - Only one claim can be approved; others are rejected with justification.
    - **Further elicitation**: Claimants are required to provide additional proof of ownership (e.g., detailed description, identifying marks, contextual evidence), and staff may request clarification before making a final decision.
- **Roles involved**: Claimants, Staff/Moderator, Admin.

CS-03: Simultaneous updates to item information

- **Scenario**: A user updates item details while another user is viewing or editing the same listing.
- **Risk**: Outdated or conflicting information being displayed or saved.
- **Expected behavior**:
    - System validates updates against the current state before applying changes.
    - If conflict exists, user is prompted to refresh or review changes.
    - Optional version tracking or logging is maintained.
- **Roles involved**: Reporters, Claimants, System.

CS-04: Item status conflict during active claim

- **Scenario**: An item is marked as returned or closed while a claim is still under review.
- **Risk**: Invalid system state and user confusion.
- **Expected behavior**:
    - System restricts status changes while a claim is active.
    - Only authorized roles (Staff/Admin) may override status with proper logging.
- **Roles involved**: Staff/Moderator, Admin.

CS-05: Conflicting or incomplete item data

- **Scenario**: Users provide inconsistent or partial information about an item.
- **Risk**: Reduced accuracy and difficulty verifying ownership.
- **Expected behavior**:
    - System enforces required fields and validation rules.
    - Staff/Admin may request clarification or additional details.
    - Incomplete entries are flagged or limited in visibility.
    - Users may be prompted to refine or correct submitted data before the report becomes fully active.
    - **Further elicitation**: The system or staff explicitly requests missing or conflicting information (e.g., clearer descriptions, additional attributes) to resolve ambiguity before accepting or promoting the report.
- **Roles involved**: Reporters, Staff/Moderator.

Implementation Notes (Cross-Team)

- Use validation and state control mechanisms to prevent inconsistent updates.
- Implement duplicate detection using key attributes such as item description, location, and time.
- Restrict critical actions (such as claims and status changes) to controlled workflows.
- Maintain logs or audit trails for updates and moderation actions when possible.
- Ensure clear user feedback when conflicts occur to maintain transparency and trust.

3.5. Risk Analysis and Mitigation

3.5.1. Risk Identification (FMEA and Guidewords)

The Lost & Found system allows users with UPRM accounts to report lost items, search reports, and contact other users to recover their items. Since the app involves user-generated content, personal contact information, and file uploads, several risks may arise and affect reliability, usability, and security. The following analysis uses FMEA and guidewords with scenarios to illustrate each risk.

Risk 1: Incorrect or False Reports

Guideword: Incorrect / False

FMEA Breakdown:

- **Failure Mode:** Users submit reports with inaccurate or intentionally false information (wrong location, description, or fabricated report).
- **Cause:** Mistaken data entry, misunderstanding fields, or malicious intent.
- **Effect Analysis:** Leads to mismatched claims, wasted administrative effort, delays in item recovery, and reduced trust.
- **Detection:** Admin review, automated validation, and cross-checking with existing reports.

Scenario:

1. A student loses a laptop and reports it in the system but selects the wrong building and incorrectly types the model.
2. Another student finds a laptop in the incorrectly listed building and submits a claim.
3. Conflicting claims trigger alerts for admin review.
4. Admin contacts both parties to verify ownership using serial numbers.
5. Owner experiences delays, frustration, and reduced confidence in the system.

Mitigation Strategy:

- Structured input fields for location and item type.
- Require detailed item identifiers (e.g., serial numbers).
- Allow edits or deletions for corrections.

- Automatic cross-checking for duplicates or conflicting reports.

Risk 2: Malicious File Uploads

Guideword: Hazardous Content

FMEA Breakdown:

- **Failure Mode:** Users upload non-image files or malicious scripts disguised as images.
- **Cause:** Intentional exploitation, lack of validation, or user confusion.
- **Effect Analysis:** Could lead to malware execution, system crashes, or unauthorized access.
- **Detection:** File validation on frontend and backend, virus scanning, and file-type checks.

Scenario:

1. A user tries to upload a `.exe` file labeled as a "photo of a lost wallet."
2. The frontend rejects the file format and notifies the user.
3. The user attempts again with a renamed `.jpeg` file containing malicious code.
4. Backend validation detects the inconsistency and quarantines the file.
5. Admin reviews the attempt and flags the user account for suspicious activity.

Mitigation Strategy:

- Strict file format restrictions (`.png`, `.jpeg`).
- Frontend and backend validation for file type and content.
- Virus scanning and automatic quarantine of suspicious files.
- User alerts and logging for security monitoring.

Risk 3: Unauthorized Access to Contact Information

Guideword: Confidentiality Breach

FMEA Breakdown:

- **Failure Mode:** Non-UPRM users attempt to access personal contact information in reports.
- **Cause:** Inadequate access controls or bypass attempts.
- **Effect Analysis:** Compromises privacy, violates data protection policies, and may facilitate harassment or fraud.
- **Detection:** Access logs, authentication checks, and role-based restrictions.

Scenario:

1. A visitor without a UPRM account browses the lost items database.
2. They attempt to click "contact owner" to retrieve personal phone/email.
3. The system denies access and shows only anonymized messages.

4. Admin monitors repeated unauthorized attempts and temporarily blocks the IP.
5. Users maintain confidence that their contact information is protected.

Mitigation Strategy:

- Require verified UPRM login to access contact info.
- Implement “View Only” mode for unverified users.
- Hide all sensitive fields unless authentication passes.
- Monitor repeated unauthorized access attempts.

Risk 4: Report Status Inconsistency

Guideword: Status Mismatch

FMEA Breakdown:

- **Failure Mode:** A report remains incorrectly marked as “Lost” after an item has been claimed or returned.
- **Cause:** System glitches, delayed updates, or concurrent modifications.
- **Effect Analysis:** Users may submit duplicate claims, admins waste time resolving conflicts, and items may be wrongly reassigned.
- **Detection:** Automated status synchronization checks and admin review.

Scenario:

1. A student reports a lost backpack.
2. Another student finds it and submits a claim.
3. Admin approves the claim, but the UI fails to refresh, showing the item as still “Lost.”
4. A third user sees the item as available and submits another claim.
5. Admin must manually resolve multiple claims, verify ownership, and update all system records.

Mitigation Strategy:

- Clear status categories (Lost, Claimed, Returned).
- Backend and frontend synchronization checks.
- Automated alerts for status conflicts.
- Logging for audit and correction purposes.

Risk 5: Duplicate Reports

Guideword: Redundancy / Confusion

FMEA Breakdown:

- **Failure Mode:** Multiple users submit separate reports for the same item.

- **Cause:** Lack of awareness of existing reports, vague descriptions, or simultaneous reporting.
- **Effect Analysis:** Creates confusion, delays resolution, and increases admin workload.
- **Detection:** Duplicate detection algorithms and manual review.

Scenario:

1. Two students find the same lost notebook and submit separate reports with slightly different descriptions.
2. The system highlights them as potential duplicates.
3. Admin reviews both reports, consolidates them, and contacts both finders.
4. One report is marked primary, the other archived, with cross-references maintained.
5. Owner receives a single notification, and the duplicate confusion is resolved efficiently.

Mitigation Strategy:

- Search and tagging functionality to reduce duplicate reporting.
- Require detailed descriptions and optional images for better matching.
- Automated duplicate detection with admin review.
- Notifications to users when potential duplicates exist.

3.5.2. Risk Assessment and Evaluation

Each risk is assessed based on its likelihood, potential impact, related competing goals, and strategies to mitigate or manage it.

Risk ID	Risk Description	Likelihood	Impact	Justification / Discussion	Mitigation Strategy	Related Competing Goals
RSK-01	Inaccurate report information leading to false matches	Medium	High	Users may provide incomplete or misleading details about lost/found items, reducing matching accuracy and trust in the system.	Implement structured reporting forms, require verification attributes, and validate inputs.	Information Visibility vs. User Privacy; Historical Completeness vs. Matching Accuracy
RSK-02	Unauthorized administrative changes compromising data integrity	Low	High	Administrators could override report states incorrectly, affecting report lifecycle consistency and trust.	Require explicit justification for overrides, maintain audit logs, notify report owners.	Administrative Flexibility vs. System Integrity

Risk ID	Risk Description	Likelihood	Impact	Justification / Discussion	Mitigation Strategy	Related Competing Goals
RSK-03	Privacy violations due to exposure of sensitive user information	Medium	High	Improper handling of private verification information could lead to identity misuse or fraudulent claims.	Enforce strict separation between public/private fields, restrict access to authorized actors, validate submissions.	Information Visibility vs. User Privacy
RSK-04	System downtime or performance degradation affecting report access	Medium	Medium	Technical failures could prevent users from submitting or retrieving reports, undermining reliability and timely recovery.	Implement redundant servers, regular backups, monitoring, and load testing.	User Autonomy vs. Workflow Consistency; Historical Completeness vs. Matching Accuracy
RSK-05	Misalignment of terminology causing confusion or errors	Low	Medium	Inconsistent naming of actors, items, or statuses could lead to misinterpretation of requirements and system outputs.	Enforce standardized terminology across documentation, system interfaces, and code.	Terminology Consistency vs. Expressiveness
RSK-06	Historical data clutter affecting search and matching relevance	Medium	Medium	Including closed or expired reports in active matching could generate false positives or irrelevant suggestions.	Exclude non-active reports from matching algorithms; retain historical data for auditing and reference.	Historical Completeness vs. Matching Accuracy

3.5.3. Risk Mitigation and Countermeasures

Each countermeasure is justified based on its expected effectiveness in reducing the likelihood or impact of the associated risk while considering system usability and stakeholder needs. The following table summarizes the candidate countermeasures for each identified risk, along with their justification and expected outcomes.

Risk ID	Candidate Countermeasure	Risk Reduction Tactic	Justification / Discussion	Expected Outcome
[RSK-01]	Enforce structured reporting forms with required descriptive attributes	Prevention	Structured inputs reduce ambiguity and increase match reliability	Higher matching accuracy; fewer false positives; improved user trust
[RSK-01]	Validate user inputs before submission	Detection	Ensures missing or inconsistent information is flagged early	Reduces likelihood of invalid reports; encourages complete information
[RSK-01]	Provide optional guidance and templates for item description	Mitigation	Helps users provide sufficient detail without overwhelming them	Improves quality of reports; reduces errors and follow-up communication
[RSK-02]	Require explicit justification for administrative overrides	Prevention	Reduces likelihood of unnecessary or arbitrary state changes	Maintains lifecycle integrity; auditability preserved
[RSK-02]	Record all administrative actions in audit logs	Detection	Allows monitoring and verification of all interventions	Traceable actions; deters improper modifications
[RSK-02]	Notify report owners when overrides occur	Mitigation	Keeps stakeholders informed and allows corrective action if needed	Increased transparency; user trust maintained
[RSK-03]	Enforce strict separation of public and private attributes	Prevention	Limits exposure of sensitive information to unauthorized actors	Privacy maintained; reduces fraudulent claims
[RSK-03]	Restrict access to verification information to authorized personnel	Prevention	Ensures that only validated actors can access sensitive data	Ownership verification remains secure; reduces misuse
[RSK-03]	Validate input fields for potential private information before submission	Detection	Detects accidental disclosure of sensitive details in public fields	Users are alerted; prevents privacy violations

3.6. Requirements Prioritization and Decision Analysis

3.6.1. Evaluation Criteria Definition

This section defines the evaluation criteria used to prioritize system features. The goal is to establish a clear and justified basis for comparing requirements before applying decision-making techniques.

Criteria Selection

The prioritization process is based on two primary evaluation criteria:

- **Value**: Represents the contribution of a feature toward achieving the system's primary objective, which is to maximize successful item recovery and improve user interaction within the Lost & Found platform.
- **Cost**: Represents the estimated implementation effort required for each feature, including development complexity, integration difficulty, and maintenance overhead.

Criteria Justification

These criteria were selected through team discussion and consensus. Each team member proposed and evaluated potential criteria, including usability, security, scalability, and reliability.

To formalize the decision, team members independently identified the criteria they considered most relevant for prioritization. The results were consolidated as follows:

Team Member	Proposed Criteria
Diego Ríos	Value, Cost, Security, Usability
Leanelys González	Value, Cost, Security
Jose Valentin	Value, Cost, Scalability
Jean Sánchez	Value, Cost, Reliability, Scalability

From this comparison, **Value** and **Cost** were the only criteria consistently selected by all team members.

After discussion, the team agreed that:

- **Value** captures the functional importance and user impact of each feature.
- **Cost** captures the feasibility and resource constraints associated with implementation.

Other criteria were considered but excluded to avoid overcomplicating the decision model. The team concluded that Value and Cost provide a balanced and sufficient framework for early-stage prioritization.

Evaluation Scale

To ensure consistent comparisons, the team adopted the standard AHP pairwise comparison scale:

- 1 – Equal contribution or cost
- 3 – Slightly more
- 5 – Strongly more
- 7 – Very strongly more
- 9 – Extremely more

Intermediate values (2, 4, 6, 8) may be used when necessary.

This scale allows team members to express relative importance in a structured and quantifiable way, forming the basis for the prioritization process described in the next section.

3.6.2. Requirements Prioritization using AHP

This section applies the Analytic Hierarchy Process (AHP) to determine the relative priority of system features based on the evaluation criteria defined previously. The analysis focuses on Epics E1 through E5.

Methodology

For both Value and Cost, pairwise comparison matrices are constructed using the AHP scale. Each matrix is then normalized to compute the relative weight of each feature. These weights are used to evaluate trade-offs and support prioritization decisions.

Team-Based Pairwise Comparison Process

To ensure that prioritization reflects the perspective of the entire team, each member independently performed pairwise comparisons of the Epics (E1–E5) using the AHP scale. Each member considered the relative importance of the Epics according to their expertise and priorities. These comparisons were then aggregated using the arithmetic mean to produce a single consensus matrix for both Value and Cost.

The following table summarizes the detailed contribution process of each team member:

Team Member	Focus Area	Contribution Details
Diego Ríos	Usability & Security	Evaluated Epics based on user interaction ease and security requirements. Rated Accessibility higher for usability, Safe Claim higher for security, and Reporting as foundational but lower risk in comparison to security-critical features.

Team Member	Focus Area	Contribution Details
Leanelys González	System Value & Feasibility	Prioritized features that maximize recovery success and overall system value. Focused on minimizing implementation difficulty while keeping high-value features prominent, particularly Reporting (E1) and Search (E2).
Jose Valentin	Scalability & Growth	Considered long-term system performance and growth. Rated Search (E2) and Admin (E5) higher to support future system scalability. Safe Claim (E3) was evaluated considering future extensibility of verification mechanisms.
Jean Sánchez	Reliability & Architecture	Emphasized system reliability, maintainability, and integration complexity. Rated Reporting (E1) and Safe Claim (E3) higher due to their dependence on robust architecture. Lower weight was assigned to Accessibility (E4) and Admin (E5) as they are supportive but less critical for reliability.

The final matrices presented in this section represent the aggregated result of all individual evaluations.

Value Analysis

Table 2. Value comparison matrix

Epic	E1 (Reporting)	E2 (Search)	E3 (Safe Claim)	E4 (Accessibility)	E5 (Admin)
E1	1	3	5	7	7
E2	1/3	1	3	5	5
E3	1/5	1/3	1	3	3
E4	1/7	1/5	1/3	1	1
E5	1/7	1/5	1/3	1	1

Key Observations:

- Reporting (E1) is the most valuable feature, as it creates the foundational data required for all other system operations.
- Search (E2) is also highly valuable, enabling users to locate relevant reports.
- Safe Claim (E3) adds important security but depends on prior matching.

- Accessibility (E4) and Admin (E5) are supportive features with lower direct impact on recovery.

Table 3. Normalized values and Relative Values

Epic	E1	E2	E3	E4	E5	Relative Value
E1	0.55	0.63	0.52	0.41	0.41	0.50
E2	0.18	0.21	0.31	0.29	0.29	0.26
E3	0.11	0.07	0.10	0.18	0.18	0.13
E4	0.08	0.04	0.03	0.06	0.06	0.05
E5	0.08	0.04	0.03	0.06	0.06	0.05

Cost Analysis

Table 4. Cost Comparison Matrix

Epic	E1 (Reporting)	E2 (Search)	E3 (Safe Claim)	E4 (Accessibility)	E5 (Admin)
E1	1	1/3	1/7	1/5	1
E2	3	1	1/3	1/3	3
E3	7	3	1	3	5
E4	5	3	1/3	1	3
E5	1	1/3	1/5	1/3	1

Key Observations:

- Safe Claim (E3) has the highest implementation cost due to complex validation and secure communication requirements.
- Accessibility (E4) also incurs significant cost due to UI/UX and compliance considerations.
- Reporting (E1) has the lowest cost, relying on straightforward data entry and storage.
- Admin (E5) and Search (E2) fall within moderate cost ranges.

Table 5. Normalized Values and Relative Costs

Epic	E1	E2	E3	E4	E5	Relative Cost
E1	0.06	0.04	0.07	0.04	0.08	0.06
E2	0.18	0.13	0.17	0.07	0.23	0.15
E3	0.41	0.39	0.50	0.62	0.38	0.46
E4	0.29	0.39	0.17	0.21	0.23	0.26
E5	0.06	0.04	0.10	0.07	0.08	0.07

Value-Cost Trade-off Analysis

Using the computed relative values and costs, a value-cost diagram is constructed to visualize feature prioritization.

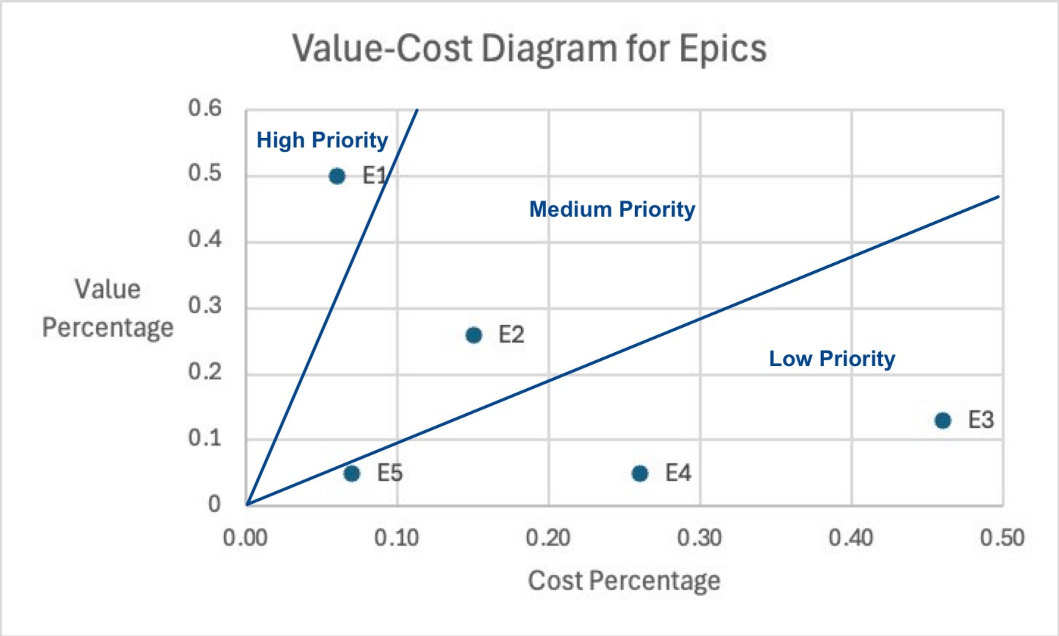

Figure 3.6.2. - 1. Value-Cost Trade-off Analysis

Final Prioritization

- **High Priority**
  - E1: Reporting Lost and Found Items — highest value with lowest cost; core system functionality.
  - E2: Search and Discovery — high value with moderate cost; essential for usability.
- **Medium Priority**
  - E3: Safe Claim and Ownership Verification — high value but also high cost; should be implemented incrementally.
- **Low Priority**
  - E4: Accessibility and Usability — important but less critical for initial deployment.
  - E5: Administrative Oversight — supports system integrity but not core recovery flow.

This prioritization supports a development strategy focused on delivering maximum impact with efficient use of resources.

3.6.3. Requirements Feasibility and Satisfiability Analysis

This section provides a detailed evaluation of the feasibility and satisfiability of selected functional requirements for the Lost & Found system. Each requirement is analyzed in terms of implementation practicality, technical considerations, and how the system can reliably meet operational objectives. Three representative requirements are considered to demonstrate this process.

Example Requirement 1: Structured Report Content

Requirement: [REQ-REPORT-F-01] – The system must accept only **Reports** that include sufficient descriptive attributes to support **Matching**, including item category, textual description, approximate location, and relevant date or time window.

Feasibility Analysis:: Implementing structured report submission is feasible using standard web and mobile form frameworks. Mandatory fields can be enforced through client-side and server-side validation, preventing incomplete submissions. The system can provide contextual help, dropdowns for categories, and geolocation-based input for location fields. Textual descriptions can be guided with character limits and examples. Backend storage using relational database tables allows indexing each attribute to optimize query performance for future searches and matching algorithms. The system must also handle partial data and support fallback mechanisms for optional fields to ensure usability without compromising feasibility.

Satisfiability Analysis:: Satisfiability is ensured by using attribute-based matching algorithms. Each report can be indexed according to category, textual keywords, location coordinates, and timestamp. This enables efficient retrieval and ranking of potential matches. Validation rules prevent inconsistent or insufficient data, ensuring that each report can participate in automated matching processes. Additionally, by separating public and private attributes (as defined in [REQ-REPORT-F-02]), the system ensures that sensitive verification data is used only for claim validation, satisfying the requirement's intent for reliability in matching.

Example Requirement 2: Report State Transitions

Requirement: [REQ-MATCH-F-02] – The system must maintain the current state of each **Report**, including Active, Returned, Closed, and Expired.

Feasibility Analysis:: Tracking report states is technically feasible using a backend state machine implementation. Each report object stores a `state` attribute, and state transitions are triggered by user actions (e.g., submission, claim verification, administrative closure) or time-based rules (e.g., automatic expiration). State transitions can be implemented using event-driven programming and database triggers to ensure atomic updates. Additional feasibility considerations include handling concurrent updates, preventing race conditions when multiple users or administrators interact with the same report, and maintaining performance under high load conditions. Audit logs can record each state transition to ensure traceability.

Satisfiability Analysis:: The system satisfies state-dependent behaviors by enforcing transition rules. For example, a report cannot move from Closed to Active, ensuring terminal states are respected. Active reports can be searched, matched, or claimed, whereas Closed or Expired reports remain read-only for historical purposes. Returned reports trigger ownership notifications and recovery confirmation workflows, ensuring that the platform's functional objectives align with the requirement. Consistency checks and automated tests can validate that each state transition behaves as intended, guaranteeing reliable execution of recovery and matching processes.

Example Requirement 3: Administrative Review

Requirement: [REQ-ADMIN-F-01] – The system must support administrative review of **Reports**.

Feasibility Analysis:: Administrative review is feasible using role-based access control (RBAC) and dedicated administrative interfaces. Administrators can query reports by status, type, or potential issues flagged by automated checks (e.g., duplicate reports or suspicious content). The backend must enforce permissions, ensuring that only authorized personnel can modify or approve reports.

Feasibility also includes providing batch processing for high-volume report review and integrating logging mechanisms to track administrative actions for accountability. Notifications or dashboards can be implemented to highlight items needing urgent attention.

Satisfiability Analysis:: Satisfiability is ensured as the system allows administrators to inspect, validate, modify, or remove reports according to institutional policies. Each action is recorded in audit trails, supporting traceability and dispute resolution. By integrating automated alerts and workflow rules, the system guarantees that reports requiring review are consistently presented to administrators. Additionally, the combination of role restrictions, audit logging, and structured workflows ensures that administrative objectives—such as content moderation and policy enforcement—are reliably achieved without compromising user access or report integrity.

Appendix

AI Disclosure Statement

Tools Used

Our team used **Artificial Intelligence** tools to help with the **Lost & Found** project. We used these tools:

- **ChatGPT (GPT-5.2 Plus tier)**: This tool helped us organize our documentation make sense of concepts like **Report**, **Ownership** and **Lifecycle State**. It also helped us write better and make our AsciiDoc files clearer. We even used it to discuss how to integrate **React** and **Supabase**.
- **GitHub Copilot**: This tool helped us write code faster by suggesting completions making **React** components and helping with **Supabase** queries and **TypeScript** types.

These tools made it easier for us to work write clearly and be organized. They didn't make any big decisions or build the system on their own.

Stages of AI Usage

We used **AI** tools at various stages of the project.

Learning & Setup

AI helped us fix problems with **React TypeScript Supabase** authentication and **Row Level Security (RLS)**. It helped us understand how to use these tools and find ways to implement them.

Brainstorming & Domain Modeling

AI helped us think of ideas for the **Lost & Found** domain, like **Report Ownership** and **Lifecycle State**. We used these ideas as starting points and made them better as a team.

Requirements & Concept Structuring

AI helped us:

- Make our requirements clearer
- Use terminology consistently
- Organize our analysis and validation sections
- Format our **PlantUML** and **AsciiDoc** sections

We reviewed and validated all requirements before including them.

Implementation Support

AI helped us:

- Plan **React** component structures

- Write **Supabase** queries and insertion logic
- Explore **RLS** policy examples
- Refine **TypeScript** interfaces

We reviewed tested and adapted all suggested code before integrating it.

Documentation

AI helped us:

- Write clearly and correctly
- Use domain terms consistently
- Organize our traceability matrices and validation sections
- Format our **AsciiDoc** and source code blocks

We reviewed edited and approved all documentation.

Human vs AI Contributions

AI tools gave us suggestions draft text and example structures.

- The team came up with all domain concepts and checked them against real **Lost & Found** scenarios.
- The team made all tech stack decisions (**React** frontend, **Supabase** backend, storage separation, **RLS** enforcement).
- The team built, tested, and debugged all integration logic.
- The team designed lifecycle transitions and moderation policies.
- The team refined all diagrams and validation scenarios.

We always reviewed, corrected, and adapted **AI**-generated material to fit the goals of the **Lost & Found** project.

There were tasks where **AI** wasn't used, like:

- Fixing state management issues
- Checking **RLS** edge cases
- Resolving authentication flow inconsistencies
- Final review for milestone alignment

Verification and Conflict Resolution

We checked all **AI**-assisted outputs carefully:

- We checked requirements against milestone guidelines.
- We referenced technical suggestions with official **React** and **Supabase** documentation.

- We tested all code suggestions locally before integrating them.
- At one team member reviewed documentation changes.

When **AI**-generated structures didn’t match our naming conventions or domain vocabulary we revised them manually.

Reflection

AI tools helped us draft faster format and brainstorm ideas. However we noticed that:

- **AI**-generated **AsciiDoc** formatting needed manual fixing.
- Some technical suggestions were general and needed domain-specific refinement.
- Human validation was crucial to ensure alignment with university **Lost & Found** workflows.

We learned that **AI** is most helpful as a productivity and drafting assistant not, as a decision-maker. It supported our work but didn’t replace critical thinking, domain understanding or collaborative decision-making.

The **Lost & Found** project is a team-driven effort. **AI** tools were one of tools used to improve efficiency and clarity while all final design, analysis, and implementation decisions were made by the team.

Logbook

Name	Section	Added/Modified
Jean P. I. Sánchez	5., 2.2.1., 2.2.3., 2.2.4., 3.2.	Modified/Added
Leanelys González Orama	1.2.1, 1.5., 2.2.1	Modified/Added
Abby Gotay Almonte	2.1.4., 2.3.1.	Modified/Added
Anthony H Martinez-Gomez	2.1.1.	Modified/Added
Lianette Alberto	1.3.2	Modified
Tasnim Said Khader	2.1.2, 2.1.3	Modified
Naedra Feliciano	3.1, 2.2.2	Added